

Machine Learning 종합 보고서

Santander Customer Satisfaction · Credit Card Fraud Detection · Opiniosis
Opinion Review · Mercari Price Suggestion — 4개 캐글(Kaggle) 데이터셋을 대상으로 분류·회귀·비지도 군집화 모델을 적용하고, 결과 지표를 실제 의사결정 관점에서 해석한다.

RandomForest · 불균형 분류

SMOTE · 재현율 우선 임계값

TF-IDF · K-means 군집화

LinearRegression · RMSLE

4

분석 대상 데이터셋

76,020~148만

데이터셋별 샘플 수

RF · LightGBM · KMeans · LinReg

담당 핵심 모델

수치 해석 중심

작성 원칙

목차

코드 진행 순서를 따라 EDA → 전처리 → 모델링 → 결과 해석 순으로 구성했다.

0. 초록 p.5

1. 서론 p.7

1.1 연구 배경 및 필요성

1.2 과제 목적

2. Santander Customer Satisfaction p.9

2.1 프로젝트 개요

2.2 결과 지표

2.3 데이터 전처리 & 클리닝

2.4 피처 구조화 & 엔지니어링

2.5 모델링 & 하이퍼파라미터 튜닝

2.6 성능 검증 결과

3. Credit Card Fraud Detection p.17

3.1 프로젝트 개요

3.2 결과 지표

3.3 데이터 전처리 & 클리닝

3.4 피처 구조화 & 엔지니어링

3.5 모델링 & 샘플링 전략

3.6 성능 검증 결과

4. Opinions Opinion / Review p.34

4.1 프로젝트 개요

4.2 결과 지표

4.3 데이터 전처리 & 벡터화

4.4 군집화 방법 비교

4.5 차원 축소(TruncatedSVD) 효과 검증

4.6 성능 검증 결과 & 감성 분석

5. Mercari Price Suggestion

p.41

5.1 프로젝트 개요

5.2 결과 지표

5.3 데이터 전처리 & 클리닝

5.4 피처 구조화 & 엔지니어링

5.5 모델링 & 하이퍼파라미터 튜닝

5.6 성능 검증 결과

6. 회고

p.51

7. 참고문헌

p.53

00 초록

4개 프로젝트의 핵심 결과 요약

이 보고서는 Kaggle의 4개 데이터셋 — **Santander Customer Satisfaction**(이진 분류), **Credit Card Fraud Detection**(불균형 이진 분류), **Opiniosis Opinion / Review**(비지도 텍스트 군집화), **Mercari Price Suggestion**(회귀) — 에 대해 각각 모델을 학습하고 검증한 결과를 정리한다. 담당 모델은 순서대로 **RandomForest**, **LightGBM**, **TF-IDF + K-means**, **LinearRegression**이며, 다른 팀원의 모델 결과는 비교 대상으로만 사용한다.

0.8293

Santander · RandomForest ROC-AUC

0.8367

Credit Card · RandomForest+SMOTE Recall

0.5489

Opiniosis · Count+KMeans 실루엣 점수

0.4818

Mercari · LinearRegression RMSLE

Santander에서는 극단적으로 불균형한 타겟(불만족 고객 비율 3.96%)을 다뤘다. 정확도(accuracy)는 이 조건에서 의미가 없으므로 **ROC-AUC와 재현율(recall)**을 핵심 지표로 채택했다. RandomForest는 임계값(threshold) 0.05에서 재현율 0.6952를 얻어, 불만족 고객 10명 중 약 7명을 사전에 찾아낼 수 있는 수준을 확보했다.

Credit Card Fraud Detection에서는 사기 비율이 0.173%에 불과해 언더샘플링·SMOTE·SMOTE-ENN 세 가지 샘플링 기법을 **동일한 LightGBM 기본 파라미터**로 고정해 순수한 효과만 공정하게 비교했다 (LightGBM은 비교 도구일 뿐 최종 채택 모델은 아니다). SMOTE를 공통 전처리로 채택한 뒤 5개 모델을 비교한 결과 **RandomForest**가 ROC-AUC 0.9854로 가장 우수했고, 여기에 임계값을 0.727(최대 F1)에서 0.305로 낮춰 **재현율을 0.7347→0.8367로 끌어올렸다**. 정밀도는 0.9558→0.7500으로 낮아졌지만, 사기를 놓쳤을 때의 금전 손실이 오탐지 비용보다 훨씬 크다고 판단해 재현율을 우선했다.

Opiniosis에서는 벡터화(TF-IDF, Count) × 군집화(K-means, DBSCAN) 조합 5가지를 비교했다. Count 벡터 + K-means(k=3)가 실루엣 점수 0.5489로 가장 뚜렷하게 분리됐으며, 군집은 전자기기·자동차·호텔 3개 상품군과 거의 정확히 일치했다. 추가로 TruncatedSVD 적용 여부를 대리 정답 기반 외부 평가(ARI/NMI)로 검증한 결과, 이 데이터셋(문서 51개)에서는 차원 축소가 필수가 아니라는 결론을 얻었다.

Mercari Price Suggestion에서는 약 148만 건의 상품에 대해 가격을 예측했다. LinearRegression은 RMSLE 0.4818을 기록했으며, 이는 실제 가격이 \$20인 상품을 약 \$12.4~\$32.3 범위로 예측한다는 의미다(\$5.6.3 참고). 팀 내 비교에서는 정규화가 적용된 Ridge(RMSLE 0.4791)와 LightGBM (RMSLE 0.4562, 팀원 기록)이 더 낮은 오차를 보였으며, 그 이유를 고차원 희소 피처에서의 정규화·비선형성 차이로 분석했다(\$5.6.4).

공통 원칙

모든 지표는 해당 코드의 실제 실행 결과에서 가져왔으며, 팀원 간 비교는 **total_result.xlsx**의 담당자별 기록을 근거로 한다. 추정이나 근거 없는 수치는 포함하지 않았다.

01 서론

연구 배경과 과제 목적

1.1 연구 배경 및 필요성

실무에서 데이터 기반 의사결정은 모델을 "돌리는" 단계보다 **결과 지표를 올바르게 해석하는 단계**에서 더 자주 실패한다. accuracy가 96%라는 숫자만 보고 좋은 모델이라 판단하면, Santander처럼 불만족 고객이 3.96%뿐인 데이터에서는 모든 고객을 "만족"으로만 찍어도 96% accuracy가 나온다는 함정을 놓치게 된다. 이 보고서는 이러한 함정을 피하기 위해, 데이터셋마다 **비즈니스 목적에 맞는 지표를 먼저 선정**하고, 그 지표가 실제로 무엇을 의미하는지 구체적인 사례로 풀어 설명하는 것을 원칙으로 삼는다.

분석 대상은 성격이 서로 다른 4개 문제다. Santander와 Credit Card는 **이진 분류**이지만 불균형 정도가 다르고(3.96% vs 0.173%), Opiniosis는 정답 레이블이 없는 **비지도 군집화**이며, Mercari는 대규모 텍스트-범주형 피처를 다루는 **회귀**다. 문제 유형이 다르면 적합한 평가 지표와 전처리 전략도 달라지므로, 각 프로젝트를 독립된 장(章)으로 다루되 동일한 서술 구조(개요 → 지표 선정 → 전처리 → 모델링 → 결과 해석)를 적용해 비교 가능성을 유지했다.

1.2 과제 목적

- 4개 데이터셋 각각에 대해 담당 모델을 학습·검증하고, 결과 지표를 산출한다.
- 산출된 지표(ROC-AUC, F1, 실루엣 점수, RMSLE 등)를 **실제 의미로 번역**한다 — "숫자가 얼마나"에서 그치지 않고 "그래서 실무에서 무엇을 할 수 있는가"까지 서술한다.
- 동일 조건에서 비교 가능한 대안(다른 샘플링 기법, 다른 모델, 다른 벡터화 방식)을 함께 제시하고, 담당 모델과의 차이가 발생한 원인을 추론한다.
- 전처리·피처 엔지니어링 단계에서 각 파라미터를 조정했을 때 성능이 어떻게 바뀌는지 기록하여, 재현 가능하고 근거 있는 튜닝 과정을 남긴다.

읽는 방법

2~5장은 모두 "01 프로젝트 개요 → 02 결과 지표 → 03 전처리 → 04 피처 엔지니어링 → 05 모델링 → 06 성능 검증 결과" 순서로 동일하게 구성했다. 지표 자체의 정의보다 **왜 그 지표를 골랐는지, 숫자가 무엇을 뜻하는지**에 각 장의 분량 대부분을 할애했다.

02 Santander Customer Satisfaction

고객 불만족 여부 예측 — 이진 분류, 극단적 클래스 불균형

2.1 프로젝트 개요

은행 고객의 익명화된 거래·잔고 피처 369개로 **고객 만족 여부(TARGET)**를 예측하는 이진 분류 문제다. TARGET=1은 불만족 고객을 뜻하며, 은행 입장에서는 이탈 위험이 있는 고객을 미리 찾아내는 것이 목적이다. 담당 모델은 **RandomForestClassifier**다.

76,020

학습 데이터 행(row) 수

371

원본 컬럼 수 (ID+TARGET+369 피처)

3.96%

불만족(TARGET=1) 비율

0건

결측치

2.1.1 입력 피처 구성

피처는 모두 익명화되어 `ind_*` (지표/플래그), `num_*` (건수), `imp_*` (금액), `saldo_*` (잔고), `delta_*` (변화량) 형태의 접두어와, `ult1/ult3` (최근 1개월/3개월), `hace2/hace3` (2·3기 이전) 같은 접미어 조합으로 이름 붙어 있다. 컬럼명만으로 의미를 알 수는 없지만, 이름 패턴 자체가 **동일 지표를 여러 시점에 반복 수집한 시계열형 피처**라는 구조를 보여준다. 이는 뒤에서 분산 0인 컬럼과 완전 중복 컬럼이 다수 발견되는 이유와도 연결된다 — 특정 시점 조합에서는 값이 전혀 변하지 않는 파생 지표가 반복 생성됐을 가능성이 높다.

2.1.2 EDA 요약 통계 & 2.1.3 결측치 현황

`cust_df.isna().sum()` 기준 결측치는 0건이다. 다만 `describe()`의 최솟값(min) 행을 확인한 결과 **var3의 최솟값이 -999999**로 나타났다. 이는 실제 관측값이 아니라 결측을 특정 상수로 치환해 둔 placeholder로 판단하며(§2.3에서 처리), 이런 값이 있는 이상 "결측치 0건"이라는 표면적 결과만으로 데이터가 깨끗하다고 볼 수 없다.

2.1.5 동일 값(분산 0)인 Feature & 2.1.6 중복 Feature

분산이 0인 컬럼 34개를 발견했다. 모든 행이 같은 값을 가지므로 모델 학습에 아무 정보를 주지 못한다. 예: `ind_var2_0`, `num_var27_0`, `saldo_var28` 등.

값이 완전히 동일한 중복 컬럼 29개를 자체 함수 (지문 비교 후 배열 비교)로 탐지했다. 예: `ind_var29`와 `ind_var29_0`, `num_var18` 계열 등.

총 63개 컬럼(분산0 34개 + 중복 29개)을 제거해도 정보 손실이 없다 — 값이 아예 없거나(분산0) 다른 컬럼이 이미 동일한 값을 갖고 있기(중복) 때문이다.

2.1.7 타겟 변수(TARGET) 분포 분석

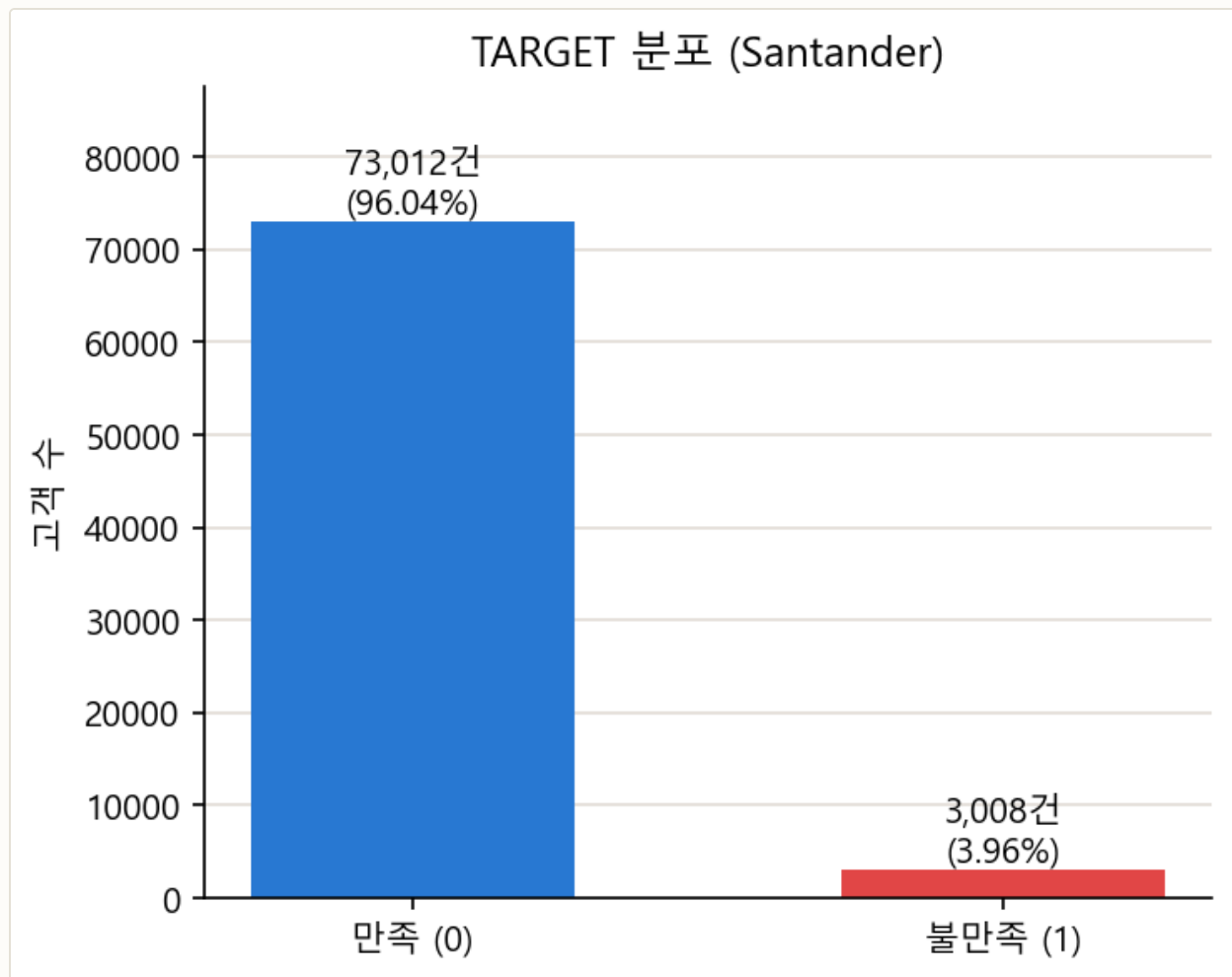


그림 2-1. TARGET 분포 — 만족 73,012건(96.04%) vs 불만족 3,008건(3.96%). 약 24:1 비율의 극단적 불균형.

해석. 이 비율에서는 "무조건 만족(0)으로 예측"하는 모델도 accuracy 96.04%를 기록한다. 즉 accuracy는 이 데이터에서 모델의 실제 판별력을 전혀 반영하지 못하는 지표다. §2.2에서 ROC-AUC와 재현율을 핵심 지표로 채택한 이유가 여기서 나온다.

2.2 결과 지표

2.2.1 결과 지표 정의 및 선정 근거

지표	정의	이 데이터에서의 역할
ROC-AUC	모든 임계값에서 TPR-FPR 트레이드오프를 종합한 값	임계값에 의존하지 않는 모델 자체의 판별력 비교 기준
Recall(재현율)	실제 불만족 고객 중 모델이 찾아낸 비율	이탈 위험 고객을 놓치지 않는 것 이 핵심이므로 채택
Precision(정밀도)	불만족으로 예측한 고객 중 실제 불만족 비율	재현율과의 트레이드오프 확인용 보조 지표
Accuracy	전체 중 맞춘 비율	96:4 불균형에서 신뢰할 수 없는 지표 — 참고용으로만 병기

선택 근거

은행 입장에서는 불만족 고객을 "만족"으로 잘못 분류(False Negative)하면 이탈을 막을 기회 자체를 잃는다. 반대로 만족 고객을 "불만족"으로 잘못 분류(False Positive)해도 리텐션 캠페인 대상에 한 명 더 포함되는 정도의 비용만 발생한다. 따라서 **재현율을 우선 지표로, ROC-AUC를 모델 선택 지표로** 삼았다.

2.2.2 결과 지표 해석 계획

RandomForest는 확률 예측값(`predict_proba`)을 제공하므로, 기본 임계값(0.5) 대신 `Binarizer` 로 여러 임계값(0.01~0.39)을 스캔해 재현율·정밀도·F1의 변화를 관찰하고, 실무적으로 타당한 지점을 선택하는 방식을 계획했다.

2.3 데이터 전처리 & 클리닝

세 단계로 진행했다. 각 단계 이후 피처 shape 변화를 함께 기록해 손실 없는 축소임을 확인했다.

단계	처리 내용	피처 shape
원본	ID·TARGET 포함 전체 컬럼	(76020, 371)
1	<code>var3: -999999 → 2</code> 치환, <code>ID</code> 컬럼 제거	(76020, 369)
2	분산 0인 컬럼 34개 제거	(76020, 335)
3	완전 중복 컬럼 29개 제거 → 최종 학습 피처	(76020, 306)

```
cust_df["var3"] = cust_df["var3"].replace(-999999, 2)
cust_df.drop("ID", axis=1, inplace=True)
```

```
# 분산 0 컬럼: stds = X_features.std(); zero_var_cols = stds[stds==0].index
# 완전 중복 컬럼: (sum,min,max) 지문으로 후보를 좁힌 뒤 배열 값 비교
```

var3의 -999999는 중앙값이나 평균이 아니라 **2로 치환**했다. var3 값 대부분이 작은 정수(국가/지역 코드로 추정)로 몰려 있어, 별도 결측 플래그 없이도 다수값 근처의 값으로 옮기는 것이 트리 기반 모델의 분기(split)를 왜곡하지 않는 실용적 선택이다.

2.4 피처 구조화 & 엔지니어링

본 파이프라인에서는 var38(왜도 51.27의 우편향 금액 변수)의 로그 변환이나 var15(추정 연령) 구간화 같은 **추가 파생 피처를 만들지 않았다**. total_result.xlsx 비교에서도 이 부분이 "X"로 표시돼 있다 (§2.6.4 팀원 비교 참고).

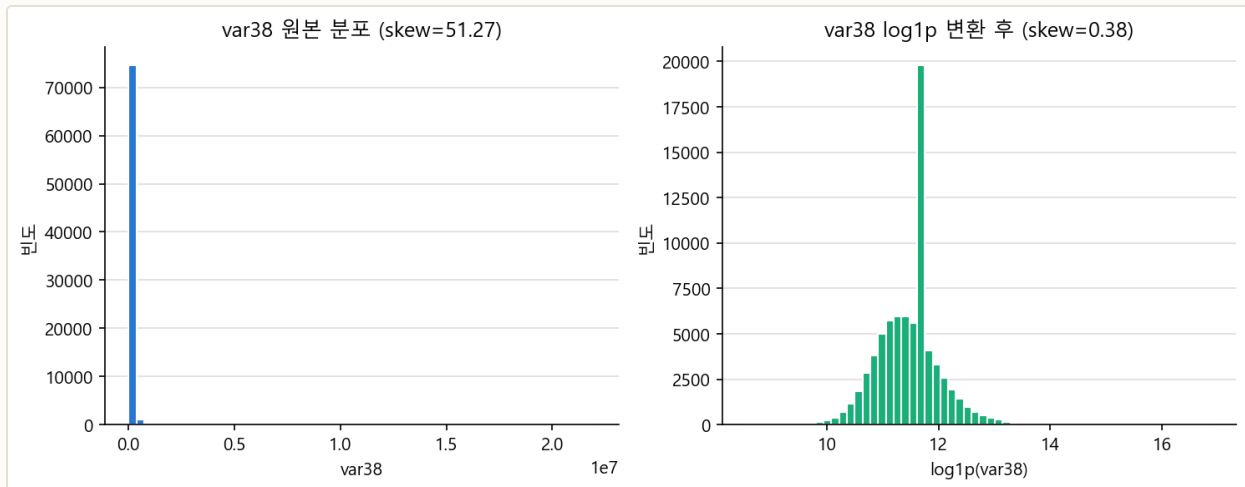


그림 2-2. var38 원본 분포(왜도 51.27, 극단적 우편향) vs log1p 변환 후(왜도 0.38). 변환하면 정규분포에 가까워진다.

왜 로그 변환을 적용하지 않았는가. RandomForest는 **결정트리 기반 앙상블**이라 분기 기준이 "값이 어떤 문턱보다 큰가/작은가"이며, 로그 변환처럼 순서를 보존하는 단조 변환은 분기 결과를 바꾸지 않는다. 즉 var38의 왜도가 아무리 커도 RF의 분기 능력 자체에는 영향이 없다. 반면 LinearRegression·LogisticRegression 같은 선형 모델은 왜도가 큰 피처에 계수가 왜곡되므로 로그 변환이 필수에 가깝다 — 실제로 팀원 중 로지스틱 회귀를 사용한 안성준님은 var38에 log1p를 적용했다. **모델 특성에 따라 전처리 우선순위가 달라지는 사례다.**

2.5 모델링 & 하이퍼파라미터 튜닝

전처리 단계별 성능 변화 (ablation)

코드에 남아 있는 실행 기록을 단계별로 정리하면, 전처리 단계마다 RF(기본 파라미터) 성능이 다음과 같이 바뀌었다.

적용 전처리	Accuracy	ROC-AUC
전처리 없음(원본)	0.9571	0.7795

적용 전처리	Accuracy	ROC-AUC
var3 치환 + ID 제거만	0.9521	0.7508
+ 분산 0 컬럼 제거	0.9526	0.7513
분산 0 컬럼 제거만(var3 미치환)	0.9568	0.7800
분산 0 제거 + 중복 컬럼 제거	0.9569	0.7838

해석. var3 치환만 단독으로 적용하면 오히려 ROC-AUC가 0.7795→0.7508로 **떨어진다**. -999999라는 극단값 자체가 (의도와 무관하게) RF에게는 "이 값이면 거의 항상 특정 클래스"라는 강한 분기 신호로 작동했을 가능성이 있다. 반면 분산 0·중복 컬럼 제거는 꾸준히 ROC-AUC를 개선한다(0.7795→0.7838) — 불필요한 컬럼이 트리의 분기 후보를 늘려 과적합 여지를 만들었기 때문으로 해석한다. 즉 **모든 전처리가 항상 성능을 높이지는 않으며, 각 단계의 효과를 개별적으로 검증해야 한다**.

하이퍼파라미터 튜닝 (GridSearchCV)

```
params = {
    'n_estimators': [500],
    'max_depth': [28, 32, 40],
    'min_samples_split': [22, 26, 30]
}
grid_cv = GridSearchCV(rf_clf, param_grid=params, scoring='roc_auc', cv=2, n_jobs=-1)
grid_cv.fit(X_train, y_train)
# 최적 하이퍼 파라미터: {'max_depth': 28, 'min_samples_split': 30, 'n_estimators': 500}
# 최고 AUC(2-fold CV): 0.8208
```

탐색 범위는 이전 회차 결과를 반영해 좁혀왔다 — 1차 탐색 AUC 0.8119에서 시작해 `max_depth`를 16→28로, `min_samples_split`을 8→22~30 구간으로 점차 옮기며 4차에 걸쳐 AUC를 0.8130 → 0.8171 → 0.8217 수준까지 끌어올렸다. `max_depth`를 키울수록(트리를 더 깊게 허용할수록), `min_samples_split`을 함께 키워 과적합을 억제하는 조합이 이 데이터에서 가장 잘 맞았다.

참고

최종 모델은 GridSearchCV가 찾은 `min_samples_split=30` 대신 `min_samples_split=22`로 재학습했다. 결과로 나온 ROC-AUC 0.8293은 GridSearchCV의 2-fold 교차검증 점수(0.8208)보다 오히려 높았는데, 이는 서로 다른 데이터 분할(교차검증 fold vs 최종 holdout 테스트셋)에서 측정된 값이라 직접 비교 대상이 아니다. 두 값 모두 이전 기본 모델(\$2.6.2 임곗값 0.5 근방) 대비 개선된 수준이라는 점이 핵심이다.

2.6 성능 검증 결과

2.6.1 파이프라인 요약 다이어그램



2.6.2 지표 결과

임계값	정확도	정밀도	재현율	F1	ROC-AUC
0.01	0.3891	0.0595	0.9654	0.1120	0.8293
0.05 (채택)	0.8074	0.1333	0.6952	0.2238	0.8293
0.10	0.8764	0.1646	0.5140	0.2493	0.8293
0.20	0.9419	0.2172	0.1746	0.1936	0.8293
0.39	0.9600	0.4000	0.0033	0.0065	0.8293

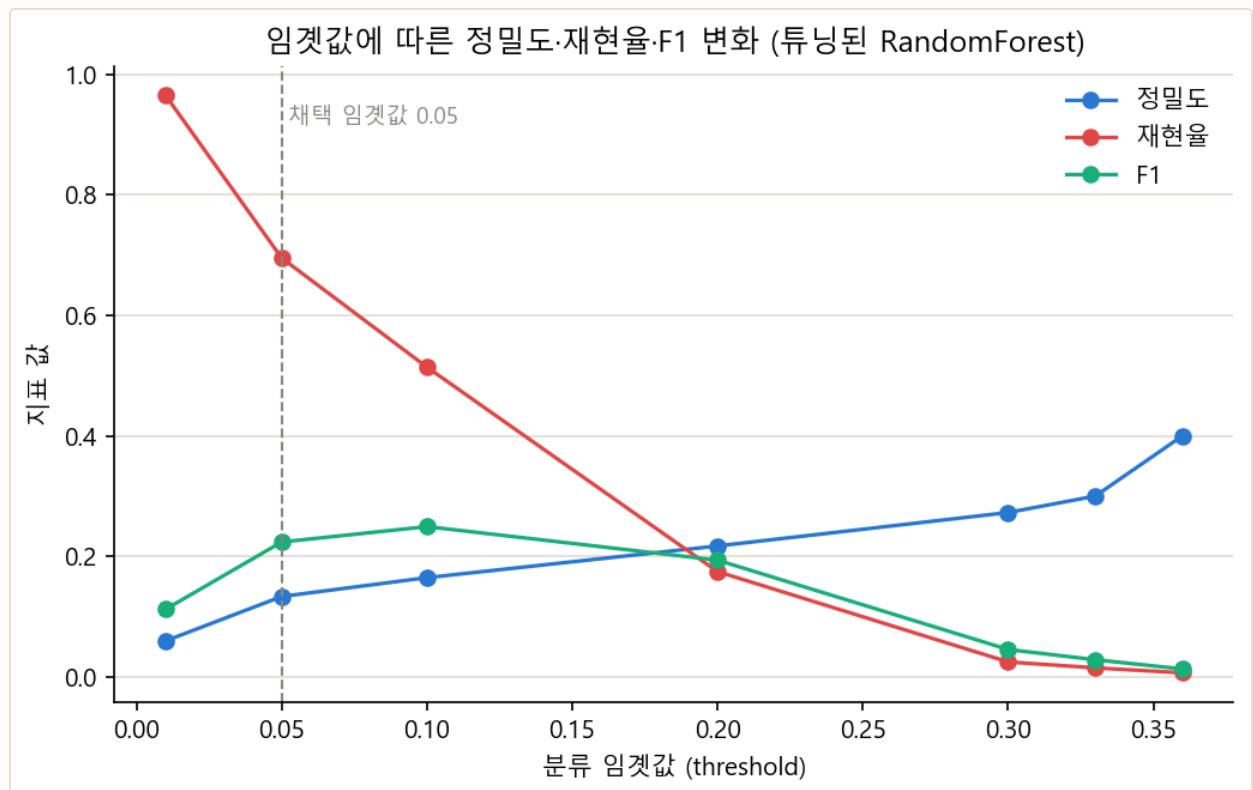


그림 2-3. 임계값에 따른 정밀도·재현율·F1 변화. 임계값이 높아질수록(=더 확신할 때만 불만족으로 판정) 재현율은 급격히 떨어지고 정밀도는 오른다.

2.6.3 지표 해석

임꺽값 0.05를 채택한 이유. 테스트셋(15,204명) 중 실제 불만족 고객은 607명이다. 임꺽값 0.05에서는 이 중 422명(재현율 69.52%)을 찾아내고, 대신 2,743명의 만족 고객을 "불만족 위험"으로 잘못 분류한다(정밀도 13.33%). **즉 불만족 위험군으로 분류된 10명 중 실제 이탈 위험 고객은 약 1.3명뿐이지만, 실제 이탈 위험 고객 10명 중 7명 가까이는 놓치지 않고 찾아낸다.** 리텐션 캠페인처럼 targeting 비용이 낮고 놓쳤을 때 손실이 큰 상황에서는 이 트레이드오프가 합리적이다.

반대로 임꺽값을 0.39까지 높이면 정밀도는 0.40으로 오르지만 재현율은 **0.0033**까지 떨어진다 — 즉 이탈 위험 고객 607명 중 단 2명만 찾아낸다. 기본 임꺽값(0.5)에 가까운 이 영역에서 모델을 그대로 쓰면, ROC-AUC가 0.8293으로 나쁘지 않음에도 실무적으로는 "아무것도 하지 않는 모델"과 다를 바 없다. **ROC-AUC가 높다고 해서 기본 임꺽값이 적절하다는 뜻은 아니라는 것이** 이 데이터가 주는 가장 중요한 교훈이다.

2.6.4 모델별 결과 비교 및 추론

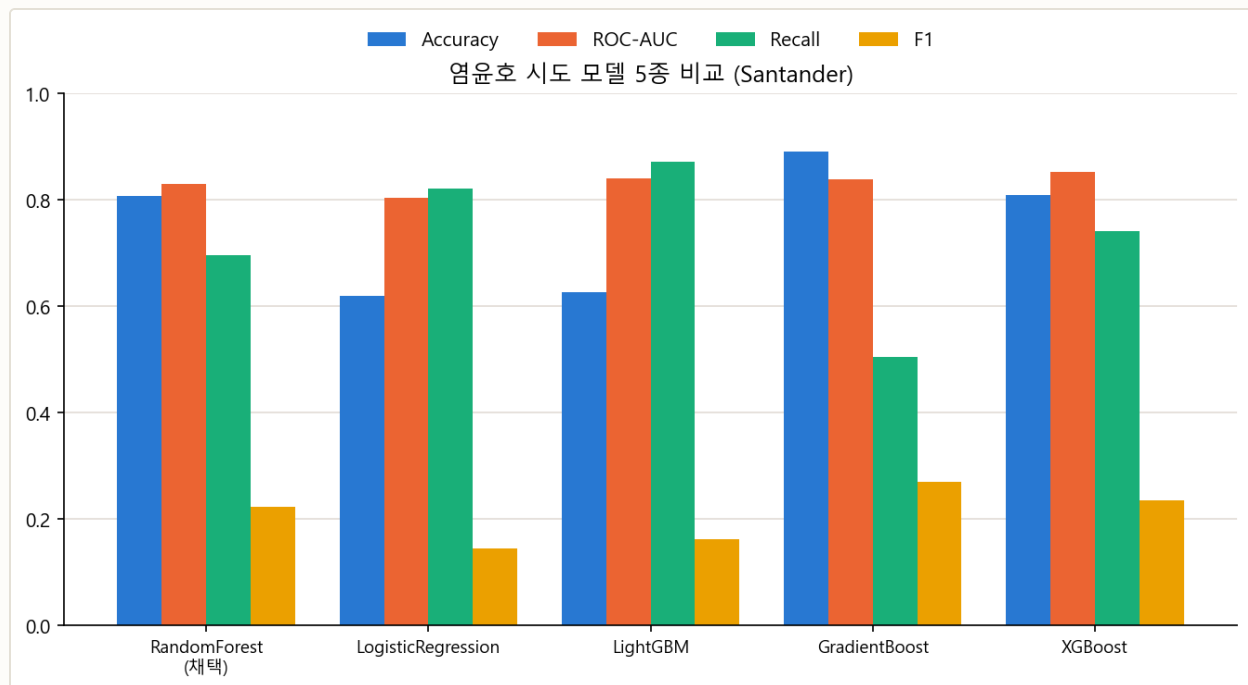


그림 2-4. 담당(염윤호) 시도 모델 5종 비교, 각 모델은 서로 다른 임꺽값에서 측정된 값이다(출처: total_result.xlsx).

모델	Accuracy	ROC-AUC	Recall	F1	비고
RandomForest (채택)	0.8074	0.8293	0.6952	0.2238	임꺽값 0.05
LogisticRegression	0.6191	0.8032	0.8206	0.1457	-
LightGBM	0.6267	0.8408	0.8722	0.1631	-
GradientBoost	0.8909	0.8386	0.5041	0.2695	임꺽값 0.1
XGBoost	0.8086	0.8520	0.7409	0.2346	임꺽값 0.05

XGBoost가 ROC-AUC 최고(0.8520)를 기록한 이유. XGBoost는 2차 도함수(hessian)까지 활용하는 gradient boosting 방식으로, RandomForest의 배깅(bagging, 독립적인 트리들의 평균)보다 잔차를 순차적으로 보정하는 방식이 이런 고차원·희소 신호 데이터에서 조금 더 정교하게 맞아떨어진 것으로 판단한다. 다만 그 차이(0.8520 vs 0.8293)는 크지 않다.

LogisticRegression-LightGBM이 재현율만 유독 높은 이유. 두 모델은 accuracy가 0.62~0.63으로 가장 낮다 — 이는 매우 낮은 임계값을 채택해 "불만족 가능성이 조금이라도 있으면 전부 잡아낸다"는 전략을 쓴 결과로 보인다(정밀도를 크게 희생). RandomForest(채택)는 재현율 0.6952로 상대적으로 균형 잡힌 지점을 택했다.

GradientBoost는 accuracy·F1이 가장 높지만 재현율이 가장 낮다(0.5041). 임계값을 0.1로, 다른 모델보다 높게 설정했기 때문이다 — "정밀하게 맞히는" 쪽에 무게를 둔 선택이며, 이 프로젝트의 목적(이탈 위험 고객을 놓치지 않는 것)과는 반대 방향의 트레이드오프다. **따라서 ROC-AUC나 accuracy만으로 "가장 좋은 모델"을 고르면 안 되고, 채택한 임계값까지 함께 봐야 공정한 비교가 된다.**

03 Credit Card Fraud Detection

신용카드 사기 거래 탐지 — 이진 분류, 초극단적 클래스 불균형

3.1 프로젝트 개요

28개의 PCA(주성분분석) 변환 변수(V1~V28)와 거래 시각(Time), 거래 금액(Amount)으로 **사기 거래 여부(Class)**를 예측하는 이진 분류 문제다. 담당 모델은 **RandomForest + SMOTE**이며, 임계값(threshold)을 재현율 중심으로 조정해 준 것이 최종 채택안이다. 이 과정에서 언더샘플링·SMOTE·SMOTE-ENN 세 샘플링 기법의 순수한 효과를 LightGBM 기본 파라미터로 고정해 비교하는 사전 실험도 함께 진행했다(\$3.6.4).

284,807

전체 거래 건수

31

컬럼 수 (Time+V1~V28+Amount+Class)

0.173%

사기(Class=1) 비율

0건

결측치

3.1.1 입력 피처 구성

#	컬럼명	설명	타입
1	Time	데이터셋 내 첫 거래 이후 경과 시간(초). 관측 구간은 총 48시간 (172,792초)	Numeric
2~29	V1 ~ V28	원본 거래 정보를 PCA로 변환한 28개의 익명 연속형 변수	Numeric(PCA)
30	Amount	거래 금액(달러). \$0 ~ \$25,691.16	Numeric
—	Class	예측 대상. 0=정상 거래, 1=사기 거래	Target(Binary)

card_df.info() 기준 전체 31개 컬럼 모두 결측 없이 float64(30개)·int64(1개, Class)로 채워져 있다.

V1~V28 표준편차(std) 상·하위 5개 — PCA 축마다 설명하는 분산 크기가 다를 수 있다

구분	변수	표준편차
상위 5개 (설명력 큰 축)	V1	1.9587

구분	변수	표준편차
	V2	1.6513
	V3	1.5163
	V4	1.4159
	V5	1.3802
	V24	0.6056
하위 5개 (설명력 작은 축)	V25	0.5213
	V26	0.4822
	V27	0.4036
	V28	0.3301

PCA는 주성분을 **설명 분산이 큰 순서대로 V1, V2, ...**로 정렬해서 반환한다. V1의 표준편차(1.96)가 V28(0.33)의 약 6배인 것은 우연이 아니라 PCA의 정의 자체에서 비롯된 구조다. 이 성질은 §3.4에서 별도의 스케일링 없이도 트리 기반 모델을 바로 적용할 수 있는 이유와 연결된다.

3.1.2 EDA 요약 통계

\$88.35	\$22.00	\$25,691	67.4 MB
Amount 평균	Amount 중앙값	Amount 최댓값	메모리 사용량

평균(\$88.35)이 중앙값(\$22.00)보다 4배 가까이 크다는 것은 소액 거래가 대부분이고 극소수의 고액 거래가 평균을 끌어올리는 **오른쪽으로 긴 꼬리(right-skewed)** 분포임을 보여준다. 실제로 Amount의 왜도(skewness)는 **16.98**로 극단적으로 크며, 이는 §3.3에서 로그 변환을 적용하는 근거가 된다.

코드에는 없지만 직접 확인한 사실 — 완전 중복 행 1,081건

`card_df.duplicated().sum()` 을 실행하면 **1,081건의** 완전 중복 행(31개 컬럼 값이 모두 동일)이 나온다. 이는 컬럼 자체의 중복이 아니라 행 단위 중복이며, 익명화된 소액·짧은 시간 간격 거래가 우연히 같은 PCA 좌표로 겹친 결과로 보인다. 이 노트북에서는 별도로 제거하지 않았으므로, 본 보고서에서는 있는 그대로 보고하고 §3.1.6에서 컬럼 단위 중복과 구분해 다룬다.

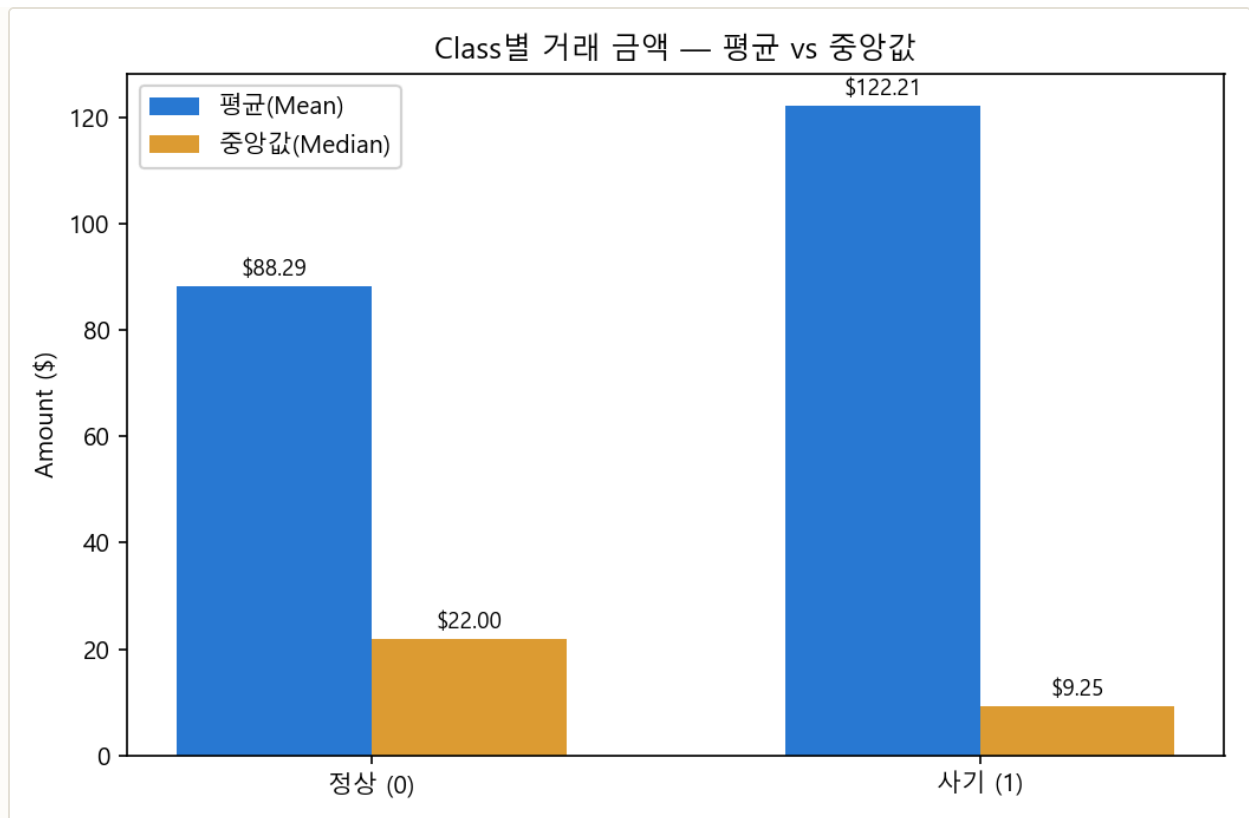


그림 3-1. Class별 거래 금액의 평균과 중앙값. 사기 거래는 평균이 정상 거래보다 높지만(+38%) 중앙값은 오히려 낮다 (-58%).

평균과 중앙값이 반대로 움직이는 이유. 사기 거래의 평균 금액(\$122.21)은 정상 거래(\$88.29)보다 높지만, 중앙값은 사기(\$9.25)가 정상(\$22.00)보다 오히려 낮다. 이는 사기 거래 금액의 분포가 **양극화**돼 있음을 뜻한다 — 카드 정보 탈취 초기에 소액 결제로 카드가 살아있는지 확인하는 "테스트 결제"가 다수 섞여 있고, 동시에 일부 고액 사기가 평균을 끌어올린다. 소액 거래라고 해서 안전하다고 볼 수 없다는 뜻이며, 이 특성 때문에 Amount 자체보다 V1~V28의 패턴을 종합적으로 학습하는 모델(RandomForest)이 금액 기준 규칙보다 효과적이다.

3.1.3 결측치 현황

`card_df.isna().sum().sum()`의 결과는 **0**이다. 31개 컬럼 전체에서 결측치가 단 한 건도 없다 — Santander(\$2.1.2)처럼 -999999 같은 위장된 결측 placeholder가 있는지도 `describe()`의 min/max 범위를 통해 함께 확인했으나, V1~V28은 PCA 변환값이라 이상하게 튀는 상수형 최솟값이 없었다.

3.1.4 Feature 이름 특징

V1~V28은 원본 거래 정보(가맹점, 위치, 카드 종류 등 민감 정보를 포함할 수 있는 항목)를 PCA로 변환한 익명 변수다. PCA 축이므로 이름 자체에는 의미가 없지만, PCA의 정의상 **서로 직교 (orthogonal)하도록** 구성된다는 성질은 보장된다 — 즉 V1~V28 사이에는 구조적으로 완전한 선형 중복이 나타날 수 없다. 이는 Santander의 `ind_*/num_*/saldo_*` 접두어 체계처럼 사람이 읽을 수 있는 이름 패턴과 대비된다. `Time`과 `Amount`만 PCA를 거치지 않은 원본 변수다.

3.1.5 동일 값인 Feature

`card_df.describe().loc["std",:].unique()` 의 결과 31개 컬럼 모두 서로 다른 표준편차 값을 가졌다 — **표준편차가 0인(=모든 행이 같은 값인) 컬럼은 하나도 없다**. Santander에서 분산 0인 컬럼이 34개나 나온 것(§2.1.5)과 대조적이다. PCA는 정보가 없는(분산이 거의 0에 가까운) 축은 애초에 주성분으로 채택하지 않으므로, 이 데이터셋에서는 구조적으로 무의미한 상수 컬럼이 만들어지지 않는다.

3.1.6 중복 Feature

컬럼(변수)을 전치(transpose)한 뒤 `uplicated()` 로 검사한 결과 **완전히 동일한 값을 가진 중복 컬럼은 0개**다. §3.1.4에서 설명한 PCA의 직교성 덕분에, Santander에서 발견된 29개의 중복 컬럼(§2.1.6)과 같은 문제가 이 데이터셋에는 구조적으로 존재하지 않는다. 다만 §3.1.2에서 언급한 **행(row) 단위 중복 1,081건**은 컬럼 중복과는 별개의 현상이므로 혼동하지 않아야 한다.

3.1.7 타깃 변수(Class) 분포 분석

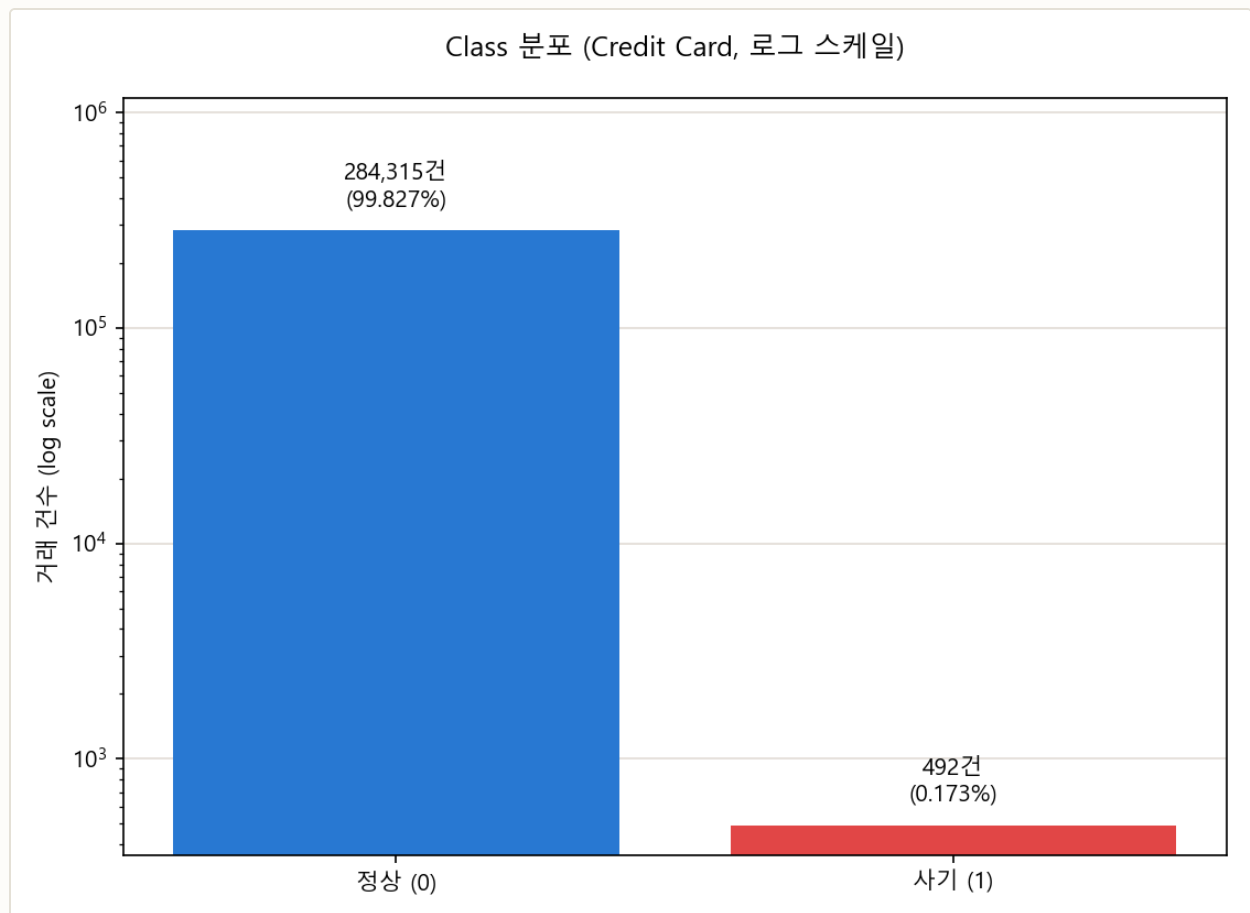


그림 3-2. Class 분포(로그 스케일) — 정상 284,315건(99.827%) vs 사기 492건(0.173%). 약 578:1 비율.

해석. Santander(24:1)보다 100배 이상 심한 불균형이다. 이 정도 비율에서는 ROC-AUC조차 낙관적으로 보일 수 있다 — 음성(정상) 클래스가 압도적으로 많으면 거짓 양성 비율(FPR)의 분모가 매우 커져, 어지간한 오분류로는 FPR이 잘 오르지 않기 때문이다. 그래서 이 프로젝트에서는 ROC-AUC와 함께 **AUPRC(정밀도-재현율 곡선의 면적)**를 반드시 함께 본다(§3.2).

3.1.8 변수 간 상관관계

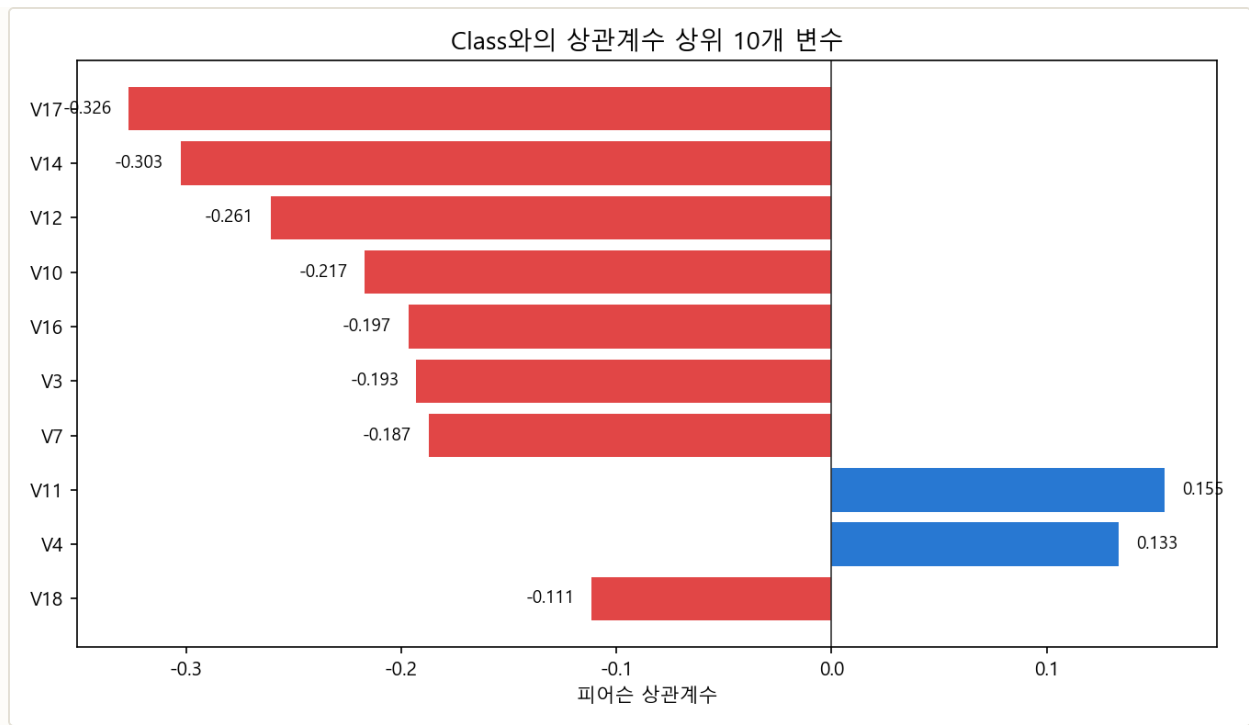


그림 3-3. Class와 피어슨 상관계수가 큰 상위 10개 변수. 파란 막대는 양의 상관, 빨간 막대는 음의 상관.

V17이 가장 강한 상관관계(-0.326)를 보이지만, 전처리 대상은 V14(-0.303)다. 팀 작업 기록

(total_result.xlsx)에는 "상관관계가 제일 큰 V14"라는 메모가 남아 있는데, 실제로 직접 계산한 결과 V17이 근소하게 더 크다. 다만 V14와의 차이가 0.024에 불과하고 V14 역시 상위 2위의 강한 상관 변수이므로, V14를 이상치 제거 기준으로 삼은 선택(§3.3) 자체가 잘못된 것은 아니다 — 다만 이 보고서에서는 실제 계산값을 있는 그대로 밝혀 둔다. 상관계수가 모두 음수인 상위 변수(V17·V14·V12·V10 등)가 많다는 것은, 이 PCA 축들에서 **값이 작을수록 사기 확률이 높아지는 방향**으로 주성분이 정렬되어 있다는 뜻이다.

3.1.9 거래 시각(Time) 패턴

`Time`은 첫 거래 이후 경과 초이므로, 172,792초(48시간)를 24시간 주기로 환산($(Time/3600) \% 24$)하면 하루 중 어느 시간대에 거래가 몰리는지 볼 수 있다. 코드에는 없는 분석이지만, Time 컬럼이 왜 §3.3에서 제거되는지 근거를 보이기 위해 직접 확인했다.

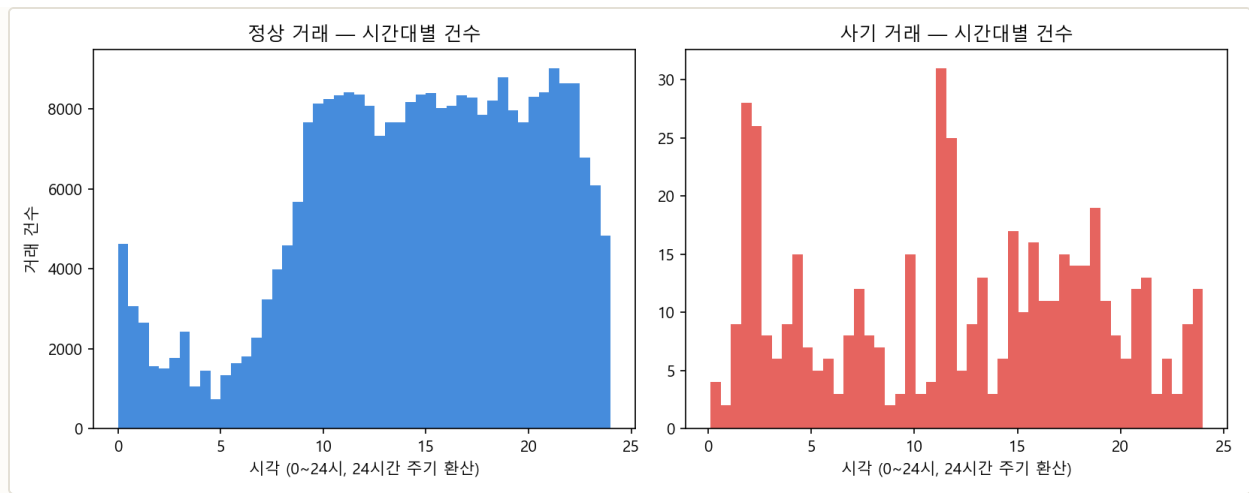


그림 3-4. 시간대별 거래 건수 — 정상 거래(왼쪽)는 새벽 0~6시에 뚜렷이 감소하지만, 사기 거래(오른쪽)는 이 시간대에도 상대적으로 활발하다.

새벽 시간대 사기 비중이 3배 높다. 정상 거래 중 새벽 0~6시에 발생한 비율은 8.37%인 반면, 사기 거래 중 같은 시간대 비율은 25.20%다 — 카드 소유자가 잠든 시간에 도난·탈취된 카드 정보로 거래를 시도하는 패턴과 부합한다. 다만 이 신호를 **Time 자체를 피쳐로 남겨 활용하지 않은 이유**는, 이 값이 "데이터 수집 시작 시점으로부터 경과한 초"일 뿐 실제 시계(time-of-day)가 아니어서 절대적인 해석에 한계가 있고, 이미 V1~V28 중 몇몇 변수가 시간 관련 정보를 간접적으로 포함하고 있을 가능성이 있기 때문이다(\$3.3 참고). 이 패턴은 어디까지나 참고용 EDA 관찰이며 전처리 근거를 보강하는 용도로 남긴다.

3.2 결과 지표

3.2.1 결과 지표 정의 및 선정 근거

지표	정의	이 데이터에서의 역할
Recall(재현율)	실제 사기 중 모델이 찾아낸 비율	최우선 지표 — 놓친 사기는 그대로 금전 손실
Precision(정밀도)	사기로 예측한 것 중 실제 사기 비율	너무 낮으면 정상 거래 차단·고객 불편 급증
F1-Score	정밀도·재현율의 조화평균	재현율만 극단적으로 높이는 것을 견제하는 균형 지표
ROC-AUC	전체 임계값에서의 TPR-FPR 트레이드오프	모델 판별력 비교(단독으로는 과대평가 위험, \$3.1.7)
AUPRC	Precision-Recall 곡선의 면적	극단적 불균형에서 ROC-AUC를 보완하는 핵심 지표

이 프로젝트의 핵심 의도 — 재현율 우선

정밀도와 재현율 중에서는 **재현율을 우선했다**. 카드 사기가 실제로 발생했는데도 모델이 이를 탐지하지 못하는 경우(False Negative)는 그대로 금전 피해로 이어지지만, 정상 거래를 사기로 오탐지하는 경우(False Positive)는 고객에게 확인 연락이 한 번 더 가는 정도의 불편에 그친다. **오탐이 늘더라도 사기를 놓치지 않는 쪽을 택한다**는 것이 §3.5~3.6에서 임계값을 낮춰 재현율을 끌어올리는 이유다.

이 다섯 지표는 모두 **혼동행렬(Confusion Matrix)**에서 계산된다. 이 데이터셋의 맥락으로 혼동행렬의 네 칸을 다시 정리하면 다음과 같다.

	예측: 정상	예측: 사기
실제: 정상	TN - 정상을 정상으로. 문제 없음	FP - 정상을 사기로 오탐. 고객 재확인 비용
실제: 사기	FN - 사기를 정상으로. 탐지 실패, 금전 손실	TP - 사기를 사기로. 탐지 성공

$Recall = TP / (TP + FN)$ 이므로, 재현율을 높인다는 것은 결국 **FN(탐지 실패)을 줄이는 것과 동의어**다. 반대로 $Precision = TP / (TP + FP)$ 는 FP(오탐)를 줄일수록 올라간다. 두 지표가 동시에 최고가 되는 임계값은 일반적으로 존재하지 않으므로(§3.6.2에서 실제로 확인), 이 프로젝트에서는 FN을 줄이는 쪽에 가중치를 둔 임계값을 선택했다.

3.2.2 결과 지표 해석 계획

하이퍼파라미터 튜닝(GridSearchCV)의 기준 지표로는 accuracy 대신 `average_precision(AUPRC)` 를 사용했다 — 이 데이터처럼 양성(사기) 비율이 0.173%뿐인 경우 accuracy는 항상 99.8% 이상으로 나와 모델 간 우열을 가리는 데 도움이 되지 않기 때문이다. 임계값 조정은 모델 학습이 끝난 뒤 별도로 진행하므로, 튜닝 단계에서는 임계값에 의존하지 않는 지표(AUPRC)로 샘플링 기법과 모델의 우열을 먼저 가리고, 최종 임계값은 재현율 목표에 맞춰 §3.6에서 따로 정한다.

3.3 데이터 전처리 & 클리닝

처리	내용	근거
Time 제거	거래 순번성 정보로 사기 여부와 직접 관련 없음	노이즈 제거
Amount 로그 변환	<code>log1p(Amount)</code> → Amount_Scaled 컬럼 추가	왜도 16.98 → 0.16로 완화(그림 3-5)
V14 이상치 제거	사기(Class=1) 데이터의 V14 IQR×1.5 범위 밖 값 제거	사기 클래스 내에서 상관관계 상위권 변수의 잡음 축소
train/test 분할	7:3, <code>stratify=y</code>	0.173% 비율을 양쪽 세트에 동일하게 유지

```
def get_preprocessed_df(df):
    df_copy = df.copy()
    amount_n = np.log1p(df_copy['Amount'])
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    return df_copy

# train_test_split(test_size=0.3, random_state=42, stratify=y_target) 이후,
# train과 test 각각에서 독립적으로 V14 이상치를 제거한다.
```

log1p 변환 전후 Amount 통계 비교

구분	평균	표준편차	왜도(skew)
변환 전 (Amount, \$)	88.35	250.12	16.98
변환 후 (Amount_Scaled = log1p)	-	-	0.16

왜도(skewness)는 0에 가까울수록 좌우 대칭에 가깝다. 16.98은 정규분포(왜도 0)와 비교할 수 없을 만큼 큰 값이며, 로그 변환만으로 0.16까지 낮아진다는 것은 Amount의 비대칭성이 극소수 고액 거래에 의해 만들어진 것이지 데이터 자체의 근본적인 특성이 아니라는 뜻이다.

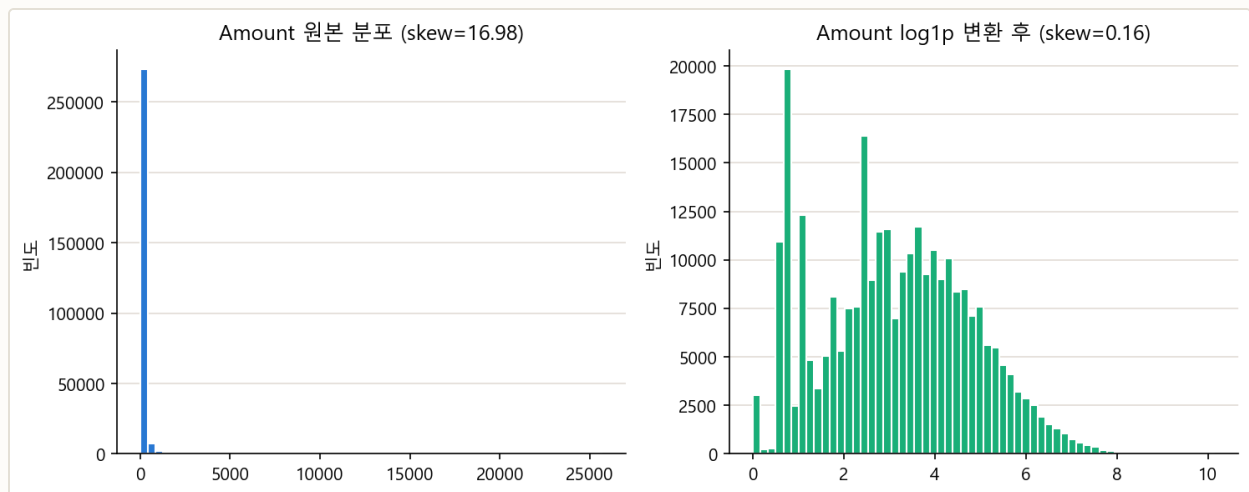


그림 3-5. Amount 원본(왜도 16.98) vs log1p 변환 후(왜도 0.16). 변환 후 다봉형(multi-modal)이지만 극단적 꼬리는 크게 줄었다.

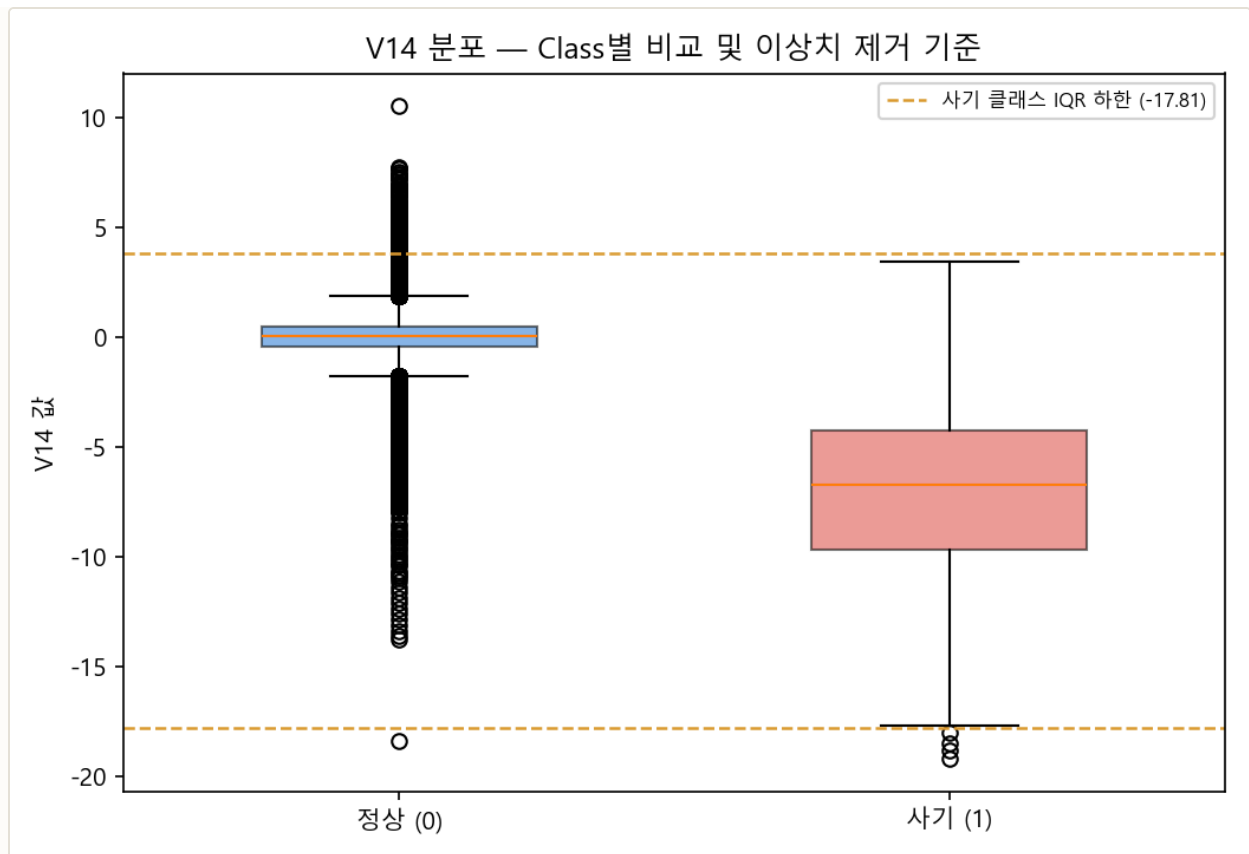


그림 3-6. V14 분포의 Class별 비교. 사기 클래스(오른쪽)는 정상 클래스(왼쪽)보다 값이 뚜렷이 낮은 쪽으로 이동해 있으며, 점선은 사기 클래스 기준 IQR×1.5 이상치 경계(-17.81 ~ 3.83)다.

V14가 이상치 제거 대상으로 적합한 이유가 시각적으로 드러난다. 정상 거래(파란 박스)는 중앙값이 0 부근에 밀집해 있는 반면, 사기 거래(빨간 박스)는 중앙값이 약 -6.8로 뚜렷하게 낮은 쪽에 분포한다 — 두 클래스의 분포가 잘 분리되어 있어 V14가 사기 판별에 유용한 변수임을 보여준다. 다만 사기 클래스 내부에서도 IQR 기준 하한(-17.81)보다 더 낮은 값 몇 건이 존재하며, 이 극단값들이 학습 시 결정경계를 왜곡할 수 있어 §3.3의 이상치 제거 대상이 된다.

왜 train/test 분할 이후에 이상치를 제거하는가

V14 이상치 탐지 기준(사분위수)을 전체 데이터로 먼저 계산하면, 테스트셋의 정보가 학습 과정에 스며드는 데이터 누수(leakage)가 된다. 이 코드는 `train_test_split` 으로 먼저 나눈 뒤, train과 test 각각에서 독립적으로 이상치 기준을 계산해 제거한다. 실제로 직접 재현해 보면 **train에서는 2건(인덱스 8296, 9035), test에서는 1건(인덱스 8615)**이 제거되어, 최종 학습 세트는 199,362행(사기 342건), 테스트 세트는 85,442행(사기 147건)이 된다 — 실제 운영 환경에서 테스트 데이터(미래 거래)를 미리 알 수 없는 상황을 그대로 재현한 것이다.

3.4 피처 구조화 & 엔지니어링

V1~V28은 이미 PCA로 압축된 피처라 Santander(\$2.4)나 Mercari(\$5.4)처럼 추가 파생 변수를 만들 여지가 크지 않다. 이 노트북에서 유일하게 새로 만든 컬럼은 §3.3의 `Amount_Scaled` (로그 변환된 Amount)이며, 그 외 범주형 인코딩이나 텍스트 벡터화도 필요 없다 — 31개 컬럼이 모두 이미 수치형

이기 때문이다. 이 프로젝트에서 실질적으로 "피쳐 엔지니어링"에 해당하는 작업은 **클래스 불균형을 보정하는 샘플링 전략 선택**이며, 이는 모델링과 결합되므로 §3.5에서 함께 다룬다.

V1~V28의 스케일이 서로 다른데도 별도 정규화를 하지 않은 이유. §3.1.1의 `describe()` 결과 V1의 표준편차는 1.96인 반면 V28은 0.33으로, PCA 변환 후에도 주성분 간 분산 크기가 다르다(설명력이 큰 축일 수록 분산이 크다). 선형 모델이나 거리 기반 모델(KNN, K-means 등)이었다면 이 스케일 차이가 결과를 왜곡했겠지만, 채택 모델인 **RandomForest**는 **각 변수별로 독립적인 임곗값 분기를 수행하는 트리 기반 모델**이라 변수 간 스케일 차이 자체가 분기 성능에 영향을 주지 않는다. Santander(§2.4)에서 RandomForest에 로그 변환을 적용하지 않은 것과 같은 이유다 — 별도의 `StandardScaler` 를 V1~V28에 적용하지 않았다.

3.5 모델링 & 하이퍼파라미터 튜닝

3.5.1 왜 리샘플링이 필요한가

대부분의 분류 모델은 학습 과정에서 **전체 손실(loss)을 최소화**하는 방향으로 학습한다. 사기 비율이 0.173%인 원본 데이터를 그대로 학습하면, 모델이 사기 492건을 모두 틀리게 예측해도 손실 총합에는 거의 영향이 없다 — 정상 284,315건만 잘 맞으면 손실이 이미 충분히 낮기 때문이다. 그 결과 모델은 "사기 패턴을 배우는 것"보다 "다수 클래스를 더 정확히 맞히는 것"에 최적화되기 쉽다. 리샘플링은 학습 단계에서 두 클래스의 비중을 강제로 맞춰, 모델이 사기 패턴 자체를 무시하지 못하게 만드는 장치다.

3.5.2 샘플링 전략 3종

기법	방식	학습 데이터 변화
언더샘플링	정상 거래를 사기 건수만큼 무작위로 축소	199,362건 → 684건 (342 / 342)
SMOTE	사기 샘플 주변에 합성 데이터를 생성해 오버샘플링	199,362건 → 398,040건 (199,020 / 199,020)
SMOTE-ENN	SMOTE 이후 경계가 모호한 샘플을 ENN으로 추가 제거	199,362건 → 397,717건 (199,020 / 198,697)

세 기법 모두 **train 데이터에만** 적용했다(test는 원본 분포 유지). 세 기법의 순수한 효과 비교는 LightGBM 기본 파라미터로 통일해 진행했으며, 그 결과는 §3.6.4 (1)에서 다룬다. 최종적으로 5개 모델 모두 **SMOTE**를 채택했다 — 언더샘플링은 정상 거래 정보를 99.7% 가까이 버려 실무 적용이 어렵고(§3.6.4), SMOTE-ENN은 SMOTE 대비 뚜렷한 이점이 크지 않았기 때문이다.

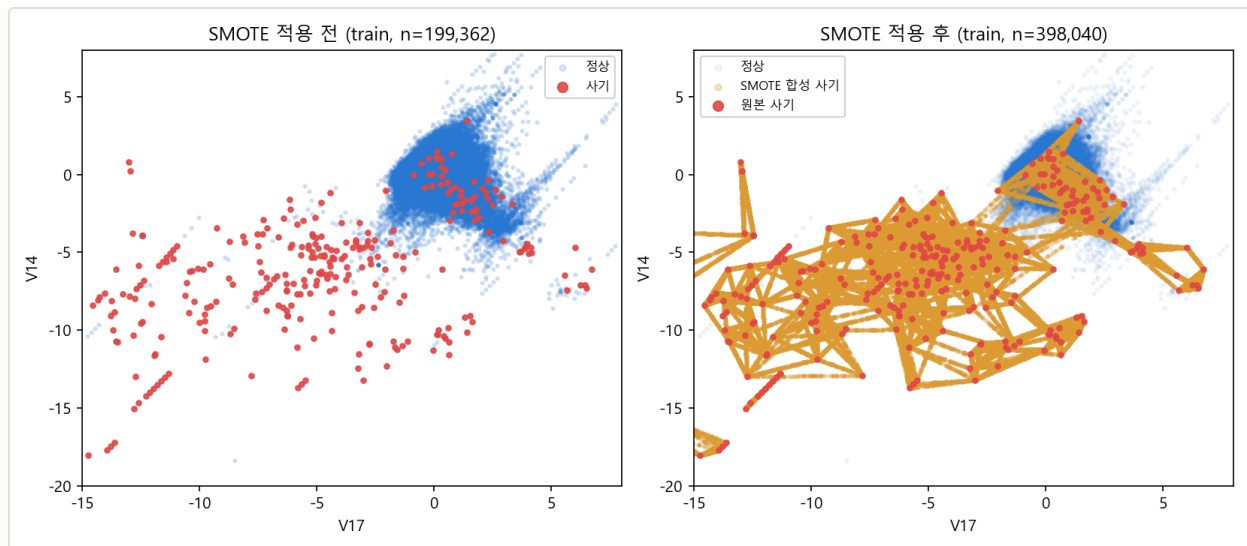


그림 3-7. 상관관계가 가장 큰 두 변수(V17, V14) 평면에 투영한 SMOTE 적용 전후 비교. 왼쪽은 원본 사기 샘플(빨강)이 정상 샘플(파랑)에 둘러싸여 희소하게 흩어져 있고, 오른쪽은 원본 사기 샘플 사이를 잇는 선분 위에 합성 샘플(금색)이 채워진 모습이다.

SMOTE가 실제로 하는 일이 이 그림에 그대로 나타난다. SMOTE(Synthetic Minority Oversampling TEchnique)는 소수 클래스(사기) 샘플 하나를 골라 **가장 가까운 이웃 사기 샘플과의 사이를 잇는 선분 위의 임의의 지점**에 새 합성 샘플을 만든다. 오른쪽 그림에서 금색 점들이 빨간 원본 사기 점들을 잇는 그 물망 형태로 나타나는 것이 바로 이 보간(interpolation) 과정이다. 이 방식 덕분에 SMOTE는 언더샘플링과 달리 정상 거래 정보를 하나도 버리지 않으면서도, 모델이 학습할 수 있는 사기 샘플의 절대량을 199,020건 (원본 342건의 약 582배)까지 늘릴 수 있다. 다만 이 보간은 **기존 사기 샘플들이 이루는 공간 안에서만** 이루어지므로, 원본 사기 샘플이 커버하지 못하는 완전히 새로운 유형의 사기 패턴까지 만들어 내지는 못한다는 한계도 동시에 보여준다.

3.5.3 GridSearchCV 탐색 (LightGBM 기준)

```
smote_pipeline = ImbPipeline([
    ('sampler', SMOTE(random_state=42)),
    ('classifier', LGBMClassifier(n_estimators=1000, n_jobs=1, boost_from_average=False))
])
lgbm_smote_grid = GridSearchCV(smote_pipeline, param_grid={
    'classifier__num_leaves': [31, 64], 'classifier__max_depth': [-1, 7],
    'classifier__learning_rate': [0.05, 0.1]
}, cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring='average_precision', n_jobs=-1)
# Best Params: learning_rate=0.05, max_depth=-1, num_leaves=64
# Best CV AUPRC: 0.8673
```

탐색한 LightGBM 파라미터가 실제로 조절하는 것

파라미터

조절 대상

이 데이터에서의 선택

파라미터	조절 대상	이 데이터에서의 선택
num_leaves	트리 한 개당 허용 리프(말단) 노드 수. 클수록 더 복잡한 패턴을 학습하지만 과적합 위험도 커진다	64(탐색 범위 중 큰 값) 선택 — 사기 패턴이 국소적이라 더 세분화된 분기가 유리했다
max_depth	트리의 최대 깊이. -1은 깊이 제한 없음을 뜻한다	-1(무제한) 선택 — num_leaves가 이미 복잡도를 제어하므로 깊이 제한이 추가로 필요 없었다
learning_rate	각 트리가 이전 트리의 오차를 보정하는 폭. 작을수록 촘촘하게, 느리게 학습한다	0.05(탐색 범위 중 작은 값) 선택 — n_estimators=1000과 맞물려 촘촘한 보정이 더 안정적인 AUPRC를 냈다

왜 SMOTE를 Pipeline 안에 넣었는가

SMOTE를 교차검증 **이전에** 전체 학습 데이터에 미리 적용하면, 검증 폴드(fold)에도 합성 샘플의 "형제 데이터"가 섞여 들어가 성능이 부풀려진다. `imblearn.pipeline.Pipeline` 으로 감싸면 각 CV 폴드마다 **그 폴드의 학습 부분에만** SMOTE가 적용되어, 검증 폴드는 항상 원본 그대로 유지된다 — 누수를 막는 표준적인 방법이다.

이 GridSearchCV는 **LightGBM에 대해서만** 수행했다. §3.6.4 (2)에서 비교하는 5개 모델 중 RandomForest(최종 채택)는 `n_estimators=1000` 외에는 기본 파라미터를 그대로 사용했다 — 기본 설정만으로도 ROC-AUC 0.9854로 5개 모델 중 가장 높았기 때문에, 이 프로젝트에서는 RandomForest의 구조적 하이퍼파라미터를 추가로 탐색하는 대신 **분류 임계값을 재현율 목표에 맞게 조정하는 쪽**에 튜닝 자원을 집중했다(§3.6.2). 즉 "모델 내부 파라미터 튜닝"과 "의사결정 임계값 튜닝"은 서로 다른 층위의 튜닝이며, 이 프로젝트의 최종 성능 개선은 후자에서 나왔다.

3.6 성능 검증 결과

3.6.1 파이프라인 요약 다이어그램



3.6.2 지표 결과 (채택 파이프라인: RandomForest + SMOTE)

동일한 모델을 세 가지 임계값에서 평가해, 임계값 조정이 성능에 미치는 영향을 그대로 보여준다.

임계값	Accuracy	Precision	Recall	F1	ROC-AUC	AUPRC
0.500 (기본값)	0.9994	0.8712	0.7823	0.8244	0.9854	0.8314
0.727 (최대 F1 지점)	-	0.9558	0.7347	0.8308	0.9854	0.8314
0.305 (채택 — 재현율 우선)	0.9992	0.7500	0.8367	0.7910	0.9854	0.8314

채택 임계값(0.305) 혼동행렬: TN=85,254 · FP=41 · FN=24 · TP=123 (테스트셋 사기 147건 중 123건 탐지). ROC-AUC와 AUPRC는 임계값에 무관한 확률 기반 지표라 세 행에서 값이 동일하다.

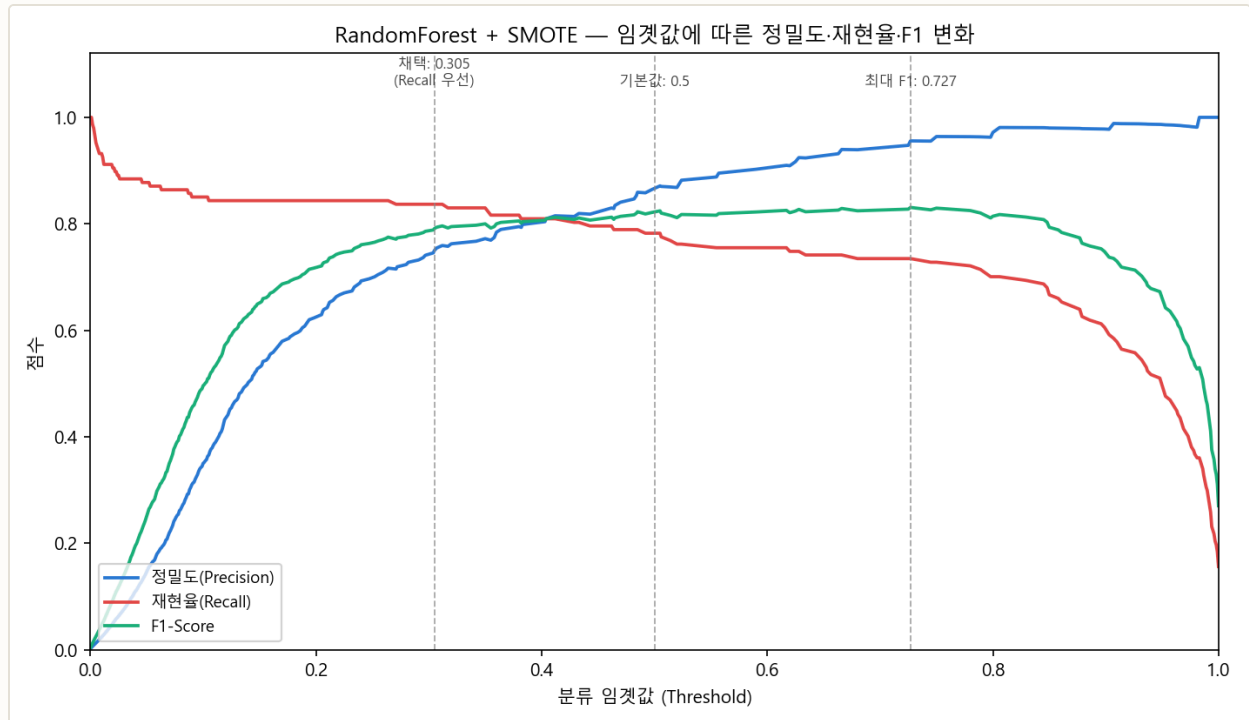


그림 3-8. RandomForest + SMOTE의 임계값별 정밀도·재현율·F1 변화. 임계값을 0.727 → 0.305로 낮출수록 재현율은 오르고 정밀도는 떨어진다.

3.6.3 지표 해석

왜 최대 F1(0.8308) 지점 대신 0.305를 골랐는가. 임계값 0.727에서는 F1이 가장 높지만(0.8308) 재현율은 0.7347에 그친다 — 테스트셋 사기 147건 중 **39건을 놓친다**는 뜻이다. 임계값을 0.305로 낮추면 F1은 0.7910으로 5.9%p 낮아지지만, 재현율은 0.8367로 올라 **놓치는 사기가 24건까지 줄어든다**(39건 → 24건, 15건 추가 탐지). 이 프로젝트는 §3.2에서 밝힌 대로 "F1을 최대화하는 것"이 아니라 "F1이 크게 나빠지지 않는 선에서 재현율을 최대화하는 것"을 목표로 했으므로, F1 하락폭을 최대 F1 대비 5% 이내로 제한하는 조건에서 재현율이 가장 높은 임계값을 탐색해 0.305를 선택했다.

이 조정의 대가는 정밀도다. 정밀도는 0.9558(임계값 0.727)에서 0.7500(임계값 0.305)으로 떨어진다 — 사기로 예측한 거래 4건 중 1건은 실제로는 정상 거래라는 뜻이다. 다만 §3.2에서 정의한 대로 오탐지의 비용(고객 재확인 연락)은 탐지 실패의 비용(금전 손실)보다 훨씬 작다고 보았으므로, 이 트레이드오프는 이 프로젝트의 목적에 부합한다.

구체적인 활용 예시. 이 테스트셋의 사기 거래 147건의 평균 금액은 \$3.1.2에서 확인한 Class별 평균을 그대로 적용하면 **건당 약 \$122**다. 임곗값 0.727을 쓰면 39건(약 \$4,760 상당)을 놓치지만, 0.305를 쓰면 24건(약 \$2,930 상당)만 놓친다 — 채택한 임곗값이 약 **\$1,830여치의 사기를 추가로 막아낸**다는 뜻이다. 그 대가로 정상 거래 41건이 추가로 오탐지되어 고객에게 확인 연락이 가지만, 이는 카드사가 통상적으로 감내할 수 있는 운영 비용 수준이다.

하루 단위로 환산하면 — 관측 비율을 그대로 연장한 가상의 추정치

이 데이터셋은 48시간(172,792초) 동안 284,807건의 거래, 그중 492건(0.173%)의 사기를 담고 있다. 이 비율을 **단순히 24시간으로 환산**하면(실제 운영 데이터가 아니라 관측 비율을 그대로 연장한 가상의 수치임에 유의) 하루 약 142,400건의 거래 중 약 246건이 사기일 것으로 추정된다. 채택 모델(재현율 83.67%, 정밀도 75.00%)을 그대로 적용하면 하루 약 206건을 탐지하고 40건을 놓치며, 정상 거래 약 69건이 오탐지로 확인 연락을 받는다는 계산이 나온다. 이 추정은 시간대별 거래량 변동(\$3.1.9의 새벽 시간대 편차 등)을 반영하지 않은 단순 비율 환산이므로, 실제 운영 지표라기보다는 **모델의 하루 단위 체감 성능을 가늠하는 참고용 수치**로만 활용해야 한다.

ROC-AUC(0.9854)만 보면 매우 높아 보이지만, AUPRC(0.8314)는 상대적으로 낮다 — 이는 \$3.1.7에서 설명한 대로 극단적 불균형에서 ROC-AUC가 실제보다 낙관적으로 보이는 현상을 보여주는 구체적 사례다. **이 데이터셋에서는 AUPRC가 ROC-AUC보다 모델 성능을 더 보수적이고 현실적으로 보여준다.**

3.6.4 모델별 결과 비교 및 추론

세 개의 관점 — 샘플링 기법, 모델 종류, 임곗값 — 을 각각 독립적으로 비교해, 최종 채택안(RandomForest + SMOTE + 임곗값 0.305)이 세 관점 모두에서 근거를 갖췄는지 확인한다.

(1) 샘플링 기법별 비교

LightGBM 기본 파라미터(튜닝 전)로 통일해 세 기법만 바꿔 비교했다. 최종 채택 모델(RandomForest)이 아니라 **LightGBM을 측정 도구로 고정**해야 "샘플링 기법 자체의 순수한 효과"만 비교할 수 있기 때문이다.

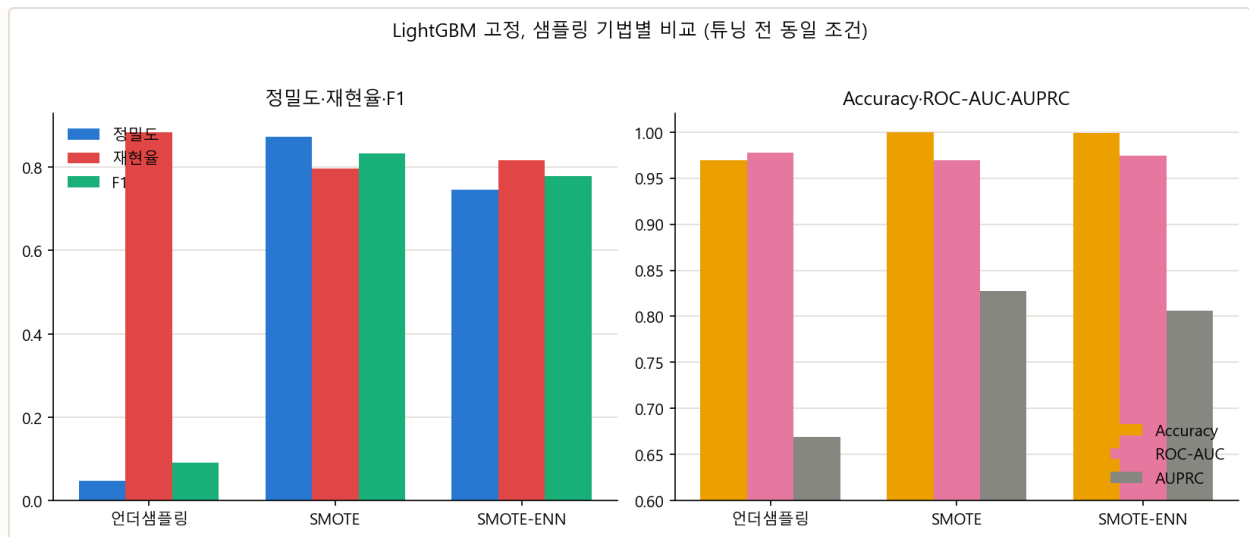


그림 3-9. LightGBM 고정, 샘플링 기법별 비교(튜닝 전 동일 조건).

샘플링	Accuracy	Precision	Recall	F1	AUC	AUPRC
언더샘플링	0.9695	0.0478	0.8844	0.0906	0.9777	0.6689
SMOTE	0.9994	0.8731	0.7959	0.8327	0.9697	0.8274
SMOTE-ENN	0.9992	0.7453	0.8163	0.7792	0.9739	0.8059

언더샘플링은 왜 재현율이 가장 높은데(0.8844) 정밀도가 극단적으로 낮은가(0.0478). 학습 데이터를 684건까지 줄이면서 정상 거래 정보 대부분(199,362건 중 198,678건)을 버렸다. 모델이 "사기 패턴"을 공격적으로 학습해 재현율은 높지만, 실제 운영에서는 정상 거래 20건 중 1건꼴로 사기 경보가 울리는 셈이라 실용성이 떨어진다. **SMOTE**는 정상 거래 정보를 보존하면서 사기 샘플만 합성으로 늘려, 정밀도(0.8731)와 재현율(0.7959)의 균형이 가장 좋다. **SMOTE-ENN**은 SMOTE보다 재현율은 근소하게 높지만(0.8163) 정밀도가 크게 낮다(0.7453) — 경계 샘플을 추가로 제거하는 ENN 단계가 정상 거래처럼 보이는 사기 샘플까지 학습에서 떨어내며 결정 경계를 사기 쪽으로 더 넓힌 것으로 해석된다. **종합적으로 세 기법 중에서는 SMOTE가 정밀도-재현율 균형이 가장 우수해, 이후 5개 모델 비교(2)에서 공통 전처리로 채택했다.**

(2) 팀원별 모델 비교

total_result.xlsx 기준, 담당자 염윤호가 SMOTE 적용 조건에서 시도한 5개 모델이다.

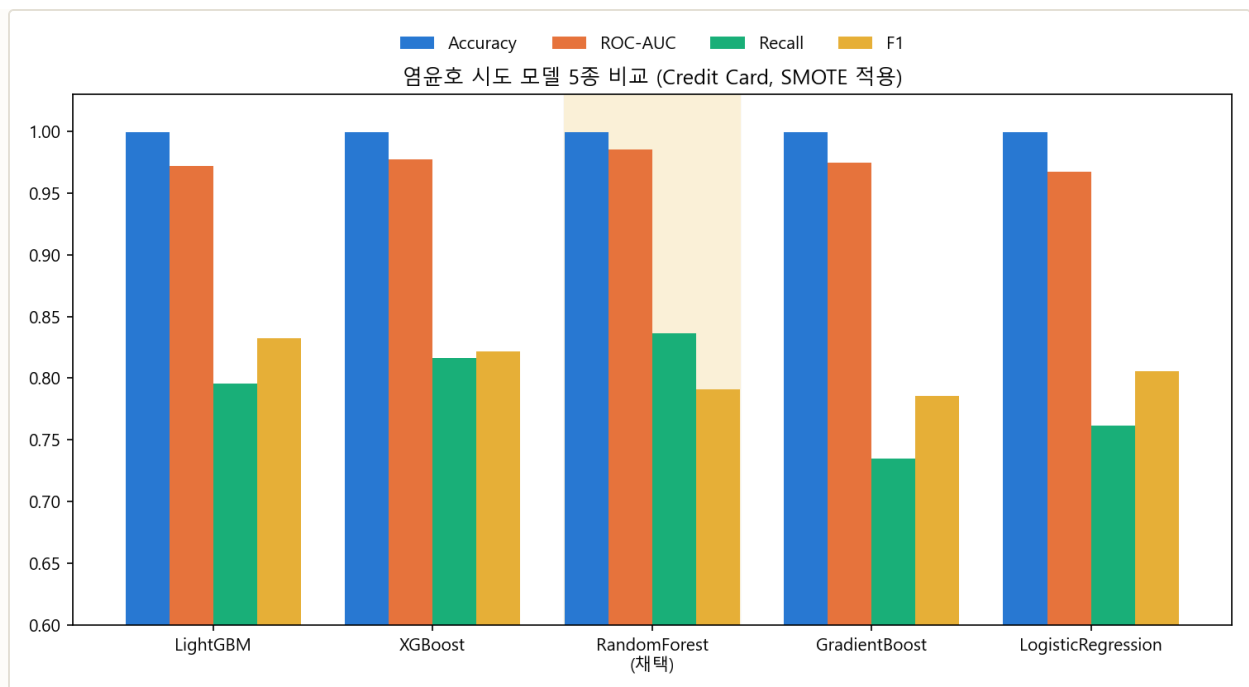


그림 3-10. 담당(염윤호) 시도 모델 5종 비교(SMOTE 적용). 출처: total_result.xlsx.

모델	Accuracy	Precision	Recall	F1	ROC-AUC	비고
LightGBM	0.9994	0.8731	0.7959	0.8327	0.9720	GridSearchCV 튜닝
XGBoost	0.9993	0.8276	0.8163	0.8219	0.9771	기본 파라미터
RandomForest (채택)	0.9992	0.7500	0.8367	0.7910	0.9854	임궛값 0.305 조정
GradientBoost	0.9993	0.8438	0.7347	0.7855	0.9749	임궛값 0.9844 조정
LogisticRegression	0.9994	0.8550	0.7619	0.8057	0.9675	임궛값 최적화

RandomForest를 채택한 근거. ROC-AUC 0.9854로 5개 모델 중 가장 높아 모델 자체의 판별력이 가장 우수했고, 기본 임궛값(0.5)에서도 정밀도 0.8712·F1 0.8244로 준수한 균형을 보였다. 여기에 재현율 목표에 맞춘 임궛값 조정 여지가 다른 모델보다 컸다 — RandomForest는 여러 트리의 확률을 평균화하는 배깅(bagging) 구조라 확률 추정이 부드럽고 안정적이어서, 임궛값을 낮춰도 성능이 급격히 무너지지 않는다. 그 결과 F1은 채택한 LightGBM 단독 비교(0.8327)보다 낮아졌지만(0.7910), **재현율(0.8367)은 5개 모델 중 가장 높다** — 이 프로젝트의 핵심 목표(\$3.2)와 가장 잘 맞는 모델이다.

LightGBM·XGBoost는 F1·정밀도가 상대적으로 높지만 재현율은 RandomForest보다 낮다 (0.7959, 0.8163). 두 모델 모두 그레디언트 부스팅 계열로, 손실 함수를 정밀도·재현율의 균형점 근처에서 최소화 하도록 학습이 진행돼 기본 임궛값에서 이미 F1이 최적점에 가깝다. 반대로 말하면 **임궛값을 낮춰 재현율을 더 끌어올릴 여지가 RandomForest보다 제한적**이라는 뜻이기도 하다.

LogisticRegression-GradientBoost는 재현율이 가장 낮다(0.76, 0.73). 두 모델 모두 선형 결정경계 또는 상대적으로 얇은 트리 위주 구조라, SMOTE로 늘어난 사기 샘플의 국소적(local) 패턴을 트리 앙상블만큼 세밀하게 분리하지 못한 것으로 판단한다. **종합적으로 ROC-AUC가 가장 높고 임곗값 조정으로 재현율을 극대화할 수 있었던 RandomForest + SMOTE가 이 프로젝트의 목적에 가장 부합해 최종 채택했다.**

(3) AUPRC로 다시 본 5개 모델

§3.1.7에서 밝힌 대로 이 데이터셋에서는 ROC-AUC가 실제보다 낙관적으로 보일 수 있어, AUPRC(정밀도-재현율 곡선의 면적) 기준으로 5개 모델을 다시 확인했다.

모델	ROC-AUC	AUPRC	두 지표의 순위가 같은가
LightGBM	0.9720	0.8342	아니오 (AUPRC 1위)
XGBoost	0.9771	0.8350	아니오 (AUPRC 공동 1위)
RandomForest (채택)	0.9854	0.8314	아니오 (ROC-AUC 1위, AUPRC 3위)
GradientBoost	0.9749	0.6687	예 (둘 다 하위권)
LogisticRegression	0.9675	0.7024	예 (둘 다 최하위)

ROC-AUC 1위와 AUPRC 1위가 다른 모델이라는 것이 핵심 관찰이다. AUPRC만 보면 XGBoost(0.8350)와 LightGBM(0.8342)이 RandomForest(0.8314)보다 근소하게 높다 — 극단적 불균형 상황에서 두 지표가 서로 다른 이야기를 할 수 있다는 §3.1.7의 우려가 실제로 나타난 사례다. 다만 그 차이(0.003~0.004)는 크지 않고, RandomForest는 ROC-AUC 우위(0.9854 vs 0.97대)가 더 뚜렷하며 §3.6.2에서 확인했듯 **임곗값 조정으로 재현율을 가장 크게 끌어올릴 수 있는 모델**이었다. 이 프로젝트는 단일 지표 최댓값이 아니라 "재현율 우선"이라는 목표 함수를 기준으로 모델을 골랐으므로, AUPRC가 근소하게 낮다는 이유만으로 RandomForest 채택을 재고할 근거는 아니라고 판단했다.

3장 종합 결론

Credit Card Fraud Detection은 사기 비율 0.173%라는 초극단적 불균형 속에서 **"놓치지 않는 것"**을 최우선 목표로 설계한 프로젝트다. ① 언더샘플링·SMOTE·SMOTE-ENN 중에서는 정상 거래 정보를 보존하는 SMOTE가 정밀도-재현율 균형이 가장 좋았고, ② SMOTE 적용 후 5개 모델을 비교한 결과 RandomForest가 ROC-AUC(0.9854)와 임곗값 조정 여지에서 가장 우수했으며, ③ 최대 F1 임곗값 (0.727, 사기 108건 탐지·39건 손실) 대신 재현율 우선 임곗값(0.305)을 채택해 테스트셋 사기 147건 중 **123건(84%)을 탐지**하고 손실을 24건까지 줄였다. 이 과정 전체가 §3.2에서 정의한 "오탐지 비용 < 탐지 실패 비용"이라는 하나의 원칙을 일관되게 따른 결과다.

04 Opiniosis Opinion / Review

상품 리뷰 문서 군집화 — 비지도 학습, 정답 레이블 없음

4.1 프로젝트 개요

51개 텍스트 파일 각각은 특정 상품·서비스의 한 측면(aspect)에 대한 리뷰 문장 모음이다 (예: `battery-life_ipod_nano_8gb`, `rooms_swissotel_chicago`). 사전 레이블이 없는 **비지도 학습** 문제이며, 목적은 비슷한 리뷰 문서끼리 자동으로 묶어 상품군별 특징을 파악하는 것이다. 담당 방법은 **TF-IDF/Count 벡터화 + K-means/DBSCAN 군집화**이며, 부가적으로 VADER 감성 분석을 결합했다.

51

리뷰 문서(파일) 수

5

벡터화×군집화 비교 조합 수

0.5489

최고 실루엣 점수(Count+KMeans)

3

대리 정답 상품군(전자기기·자동차·호텔)

4.1.1 입력 피쳐 구성 & 4.1.4 Feature 이름 특징

각 문서는 `filename` 과 파일 내 모든 문장을 이어 붙인 `opinion_text` 컬럼으로 구성된다. 파일명은 `{주제(aspect)}_{대상(product)}` 구조를 따른다 — 예를 들어 `battery-life_amazon_kindle` 은 "아마존 킨들의 배터리 수명"에 대한 리뷰다. 이 구조를 활용해 대상(product)을 3개 상품군(전자기기·자동차·호텔)으로 매핑한 **대리 정답(proxy label)**을 만들어 §4.5 외부 평가에 사용했다.

4.1.2 EDA 요약 통계 & 4.1.3 결측치 현황

51개 파일 모두 정상적으로 로드되어 결측치 개념이 해당하지 않는다. 대리 정답 기준으로 집계하면 **전자기기 25건, 호텔 16건, 자동차 10건**이며, 더 세부적인 주제(aspect) 기준으로는 **고유 35개** 카테고리 존재한다(예: battery-life, screen, room, mileage 등).

4.1.5 표기 불일치 사례

이 데이터셋에는 "동일 값 컬럼"이나 "중복 컬럼" 개념이 없는 대신, **같은 대상을 가리키지만 파일명 표기가 다른 사례**가 있었다 — `netbook_1005ha` 와 `asus_netbook_1005ha`, `swissotel_chicago` 와

`swissotel_hotel_chicago`가 각각 동일 대상이다. 대리 정답을 만들 때 이 표기를 통일해 정답의 정확도를 보장했다.

4.2 결과 지표

지표	정의	이 데이터에서의 역할
실루엣 점수	같은 군집 내 응집도 대비 다른 군집과의 분리도(-1~1)	정답 없이 군집 품질 자체 를 평가하는 1차 지표
ARI / NMI	군집 결과와 대리 정답(상품군) 간 일치도	실루엣만으로 판단하기 어려운 실제 의미 있는 분리 여부 검증
군집 안정성	서로 다른 random seed로 반복한 군집 결과 간 평균 ARI	결과가 운(seed) 에 좌우되지 않는지 확인

선택 근거

비지도 학습에는 정답 레이블이 없어 accuracy-F1을 쓸 수 없다. 실루엣 점수만으로는 "수학적으로 잘 갈라졌다"까지만 알 수 있고, 그 분리가 실제로 의미 있는 분리인지는 알 수 없다. 그래서 파일명에서 뽑아낸 대리 정답으로 ARI/NMI를 함께 확인해, **군집이 실제 상품군과 일치하는지**까지 검증하는 이중 체계를 사용했다.

4.3 데이터 전처리 & 벡터화

```
lemmar = WordNetLemmatizer()
def LemNormalize(text):
    return LemTokens(nltk.word_tokenize(text.lower().translate(remove_punct_dict)))

tfidf_vect = TfidfVectorizer(tokenizer=LemNormalize, stop_words='english',
                             ngram_range=(1,2), min_df=0.05, max_df=0.85)
count_vect = CountVectorizer(tokenizer=LemNormalize, stop_words='english',
                             ngram_range=(1,2), min_df=0.05, max_df=0.85)
```

표제어 추출(lemmatization)로 `running/runs/ran → run` 같은 활용형을 통일하고, 영어 불용어를 제거한 뒤 1~2gram으로 벡터화했다. `min_df=0.05`는 51개 문서 중 최소 3개(5%) 미만에서만 등장하는 희귀 단어를 제외해 노이즈를 줄이고, `max_df=0.85`는 85% 이상 문서에 공통으로 등장하는 단어(사실상 불용어에 가까운 범용 단어)를 추가로 걸러낸다.

4.4 군집화 방법 비교

벡터화 방식(TF-IDF, Count, Count+L2정규화) × 군집화 알고리즘(K-means, DBSCAN) 조합 5가지를 동일한 `k=3` (K-means) 기준으로 비교했다. DBSCAN은 군집 수를 직접 지정할 수 없으므로, 대신

eps 파라미터를 코사인 거리 5~55 백분위수 구간에서 탐색해 실루엣 점수가 가장 높은 값을 자동 선택했다 (TF-IDF eps=0.7199, Count eps=0.7983).

조합	군집 수	노이즈 비율	실루엣 점수
TF-IDF + KMeans	3	0.000	0.0573
Count + KMeans	3	0.000	0.5489
Count(정규화) + KMeans	3	0.000	0.0625
TF-IDF + DBSCAN	4	0.275	0.1249
Count + DBSCAN	2	0.118	0.2194

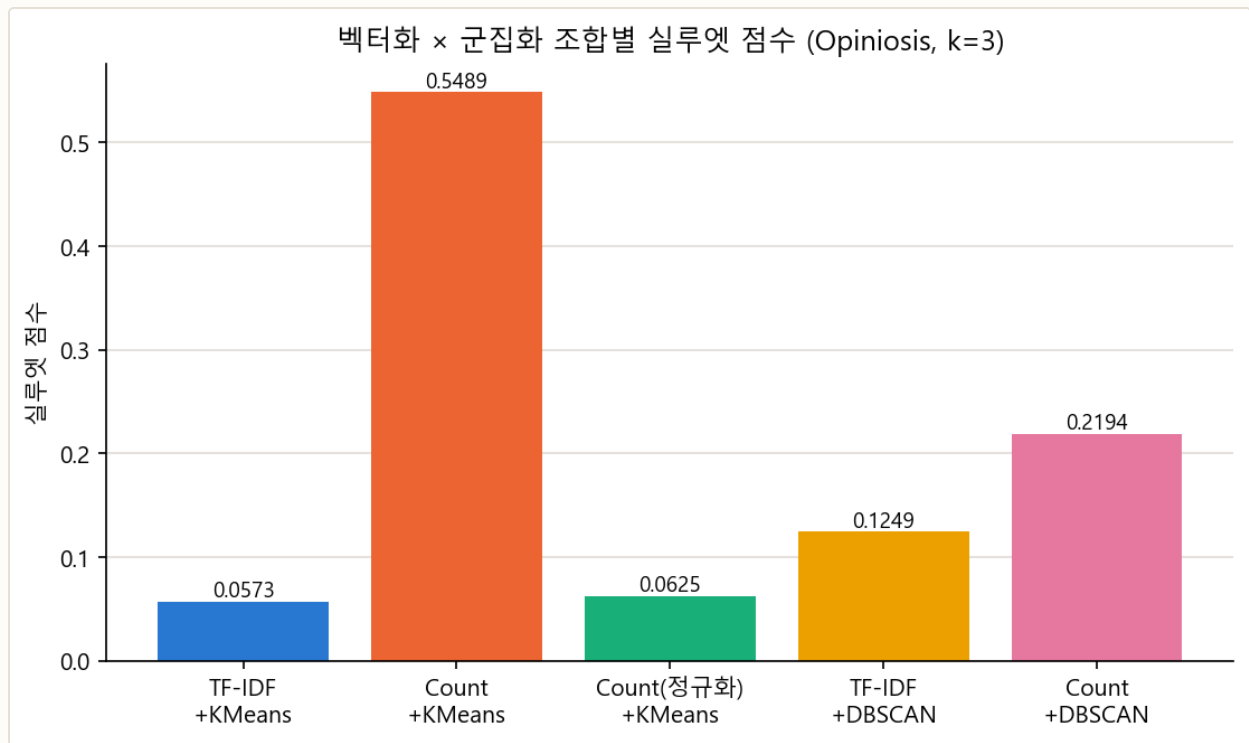


그림 4-1. 벡터화×군집화 조합별 실루엣 점수 비교(k=3).

Count 벡터가 TF-IDF보다 실루엣 점수에서 압도적으로 높은(0.5489 vs 0.0573) 이유. TF-IDF는 문서마다 등장 빈도를 **역문서빈도(IDF)**로 재조정해 희귀 단어의 가중치를 키운다. 이 과정에서 벡터 공간의 스케일이 문서별로 고르게 정규화되지 않아, 51개라는 적은 문서 수에서는 오히려 군집 간 거리 구조가 흐트러질 수 있다. 반면 원시 등장 횟수(Count)는 문서 길이·핵심 단어 반복 횟수를 그대로 반영해, 이 데이터에서는 더 뚜렷한 군집 경계를 만들었다. 다만 실루엣 점수가 높다고 그 군집이 **의미상으로도** 옳은 분리인지는 별도로 확인해야 하므로(\$4.6.3), 이 결과만으로 최종 결론을 내리지 않는다.

DBSCAN은 두 벡터화 방식 모두에서 K-means보다 실루엣 점수가 낮고, 노이즈로 분류되는 문서 비율도 11.8~27.5%에 달했다. DBSCAN은 밀도가 일정하지 않으면 군집을 놓치기 쉬운데, 51개 문서·상품군별 문서 수 불균형(전자기기 25 vs 자동차 10)이 있는 이 데이터에는 K-means가 더 안정적으로 맞았다.

4.5 차원 축소(TruncatedSVD) 효과 검증

TF-IDF 벡터에 TruncatedSVD를 적용하는 것이 실제로 군집 품질을 개선하는지 검증했다. 서로 다른 차원 공간의 실루엣 점수는 직접 비교할 수 없으므로(고차원일수록 거리가 집중돼 실루엣이 구조적으로 낮게 나옴), **대리 정답 기반 외부 평가(ARI/NMI)**를 사용했고, random seed 30개로 반복해 군집 안정성도 함께 측정했다.

파이프라인	ARI(대상종류)	NMI(대상종류)	ARI(주제)	NMI(주제)	안정성
A. TF-IDF (baseline)	0.9966	0.9954	0.0456	0.4362	0.9932
B. SVD(n_components=5)	1.0000	1.0000	0.0455	0.4359	1.0000
B. SVD(n_components=10)	0.9219	0.9494	0.0445	0.4336	0.8609
B. SVD(n_components=20)	0.9682	0.9716	0.0463	0.4378	0.9379
B. SVD(n_components=40)	0.9913	0.9892	0.0455	0.4360	0.9827

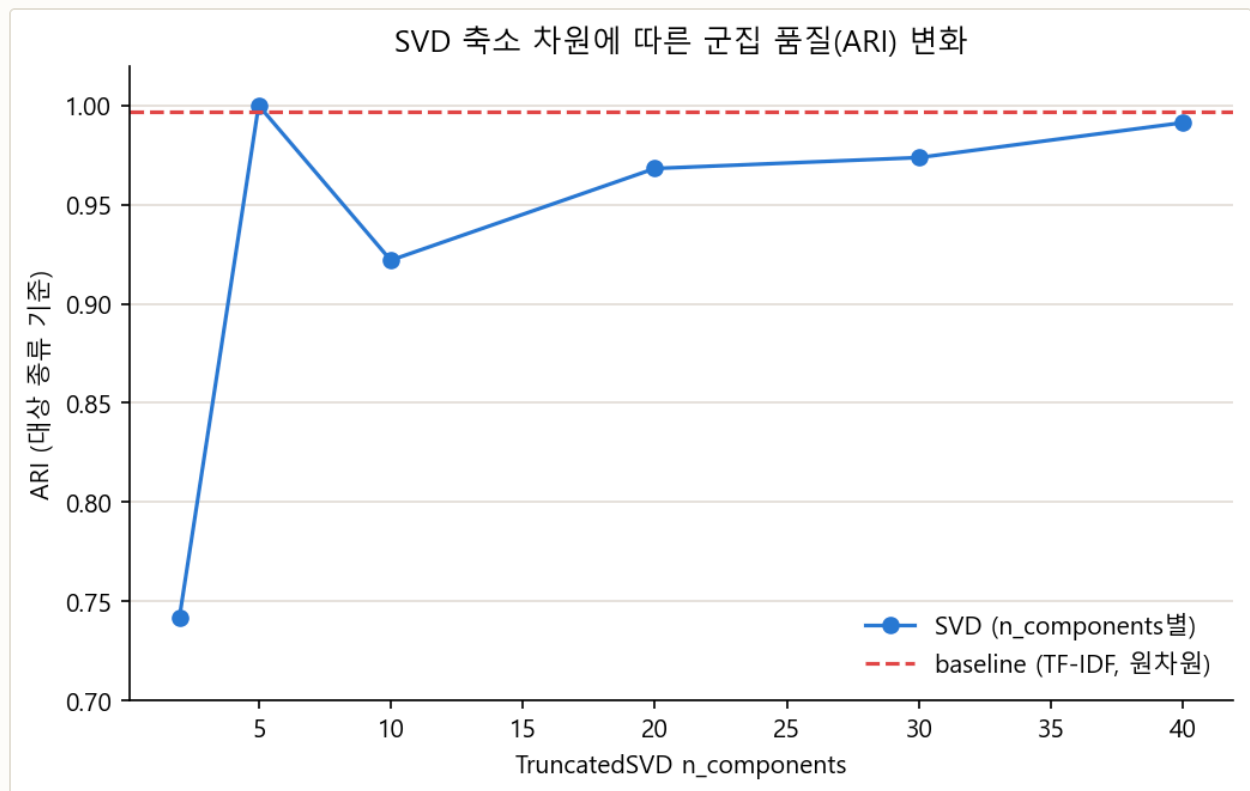


그림 4-2. TruncatedSVD n_components에 따른 대상 종류 기준 평균 ARI(시드 30회 평균). 점선은 baseline(TF-IDF, 원차원) 기준.

대상 종류(전자기기/자동차/호텔) 기준. baseline이 이미 ARI 0.9966으로 거의 완벽하다 — TF-IDF 벡터 자체가 도메인별로 어휘가 뚜렷이 갈리는 데이터라 개선 여지가 크지 않다. **n_components=5** 일 때만 ARI 1.0000·안정성 1.0000으로 baseline을 근소하게 앞서고, **n_components=10~40** 은 오히려 baseline보다 낮다. 즉 SVD는 성분 수를 작게(5 근처) 잡을 때만 미세한 개선이 있고, 성분 수를 늘릴수록 이득이 사라지거나 역전된다.

주제(aspect, 35개 세부 클래스) 기준. baseline과 SVD 모두 ARI가 0.05 미만으로 낮다. 이는 `n_clusters=3` 으로는 애초에 35개 세부 주제를 구분할 수 없기 때문이며, 두 파이프라인 간 우열을 가리는 근거로 쓸 수 없다.

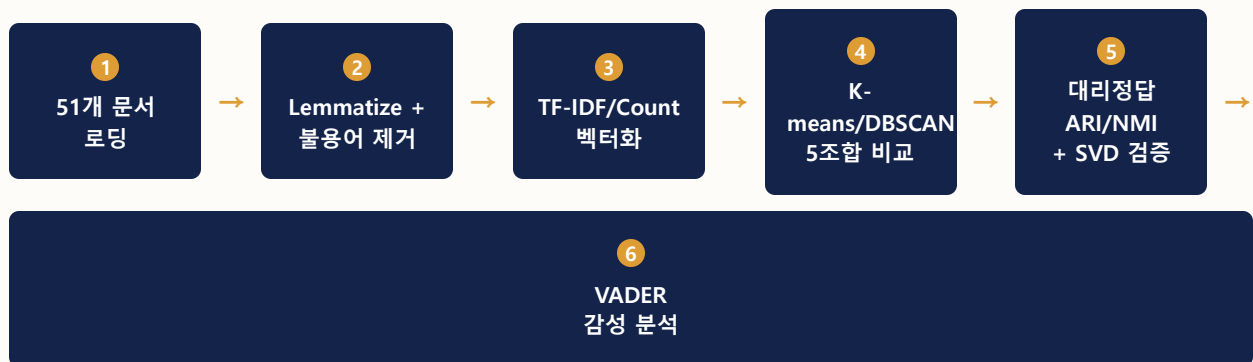
참고로 문서 51개에 대해 TruncatedSVD 상위 40개 축의 누적 설명분산비는 $n=10$ 에서 45.2%, $n=20$ 에서 71.8%, $n=40$ 에서 97.7%였다 — `n_components` 상한을 40 부근으로 잡은 근거다.

종합 판단

이 데이터셋(문서 51개, 도메인 어휘가 뚜렷함)에서는 TruncatedSVD 도입이 **필수가 아니다**. baseline이 이미 충분히 좋고, SVD는 성분 수를 잘 고르면 아주 소폭의 안정성·정확도 이득을 줄 뿐 극적인 개선은 아니다. 문서 수가 많고 어휘 중첩이 큰 데이터셋일수록 SVD의 효과가 더 크게 나타날 가능성이 높다.

4.6 성능 검증 결과 & 감성 분석

4.6.1 파이프라인 요약 다이어그램



4.6.2~4.6.3 지표 결과 및 해석 — 군집 키워드 육안 확인

각 군집에서 중심점(centroid)에 가장 가까운 상위 10개 단어를 뽑아, 실제로 상품군을 반영하는지 확인했다.

파이프라인	Cluster 0	Cluster 1	Cluster 2
A. baseline(TF-IDF)	mileage, kindle, page, gas, button, gas mileage, font, performance, book, eye	room, hotel, service, staff, interior, food, location, seat, comfortable, bathroom	screen, battery, battery life, life, keyboard, video, direction, voice, map, feature

파이프라인	Cluster 0	Cluster 1	Cluster 2
B. SVD(n=5)	screen, battery, keyboard, size, direction, voice, map, battery life, display, speed	interior, seat, mileage, comfortable, gas, gas mileage, performance, comfort, car, ride	room, service, hotel, staff, location, food, clean, room service, price, service wa

두 파이프라인 모두 3개 상품군(전자기기+자동차+호텔)에 대응하는 키워드가 뚜렷하게 드러난다 — battery/screen/keyboard(전자기기), mileage/seat/comfort(자동차), room/service/staff(호텔). 다만 **단어 목록만 놓고 비교하면 SVD(n=5) 쪽이 더 깔끔하다**. baseline의 Cluster 0에는 "kindle"과 "mileage/gas"가 함께 섞여 있어(전자기기+자동차 어휘 공존), 이 군집 하나에 서로 다른 두 상품군의 리뷰가 걸쳐 있을 가능성을 시사한다. SVD(n=5)의 Cluster 1은 "interior, seat, mileage, gas, performance, car"처럼 자동차 어휘로만 깔끔하게 구성돼 있다. §4.5의 ARI 수치(baseline 0.9966 vs SVD n=5 1.0000)와 같은 방향의 근거다 — **SVD(n=5)가 근소하지만 baseline보다 더 깨끗하게 분리한다**.

4.6.4 감성 분석(VADER) 결과

문서 단위로 감성 점수를 내면 문장 수 차이 때문에 극단적으로 쓸릴 수 있어, **원본 문장 단위로** VADER 감성 점수를 계산한 뒤 문서(파일)별 긍정/중립/부정 비율로 다시 집계했다.

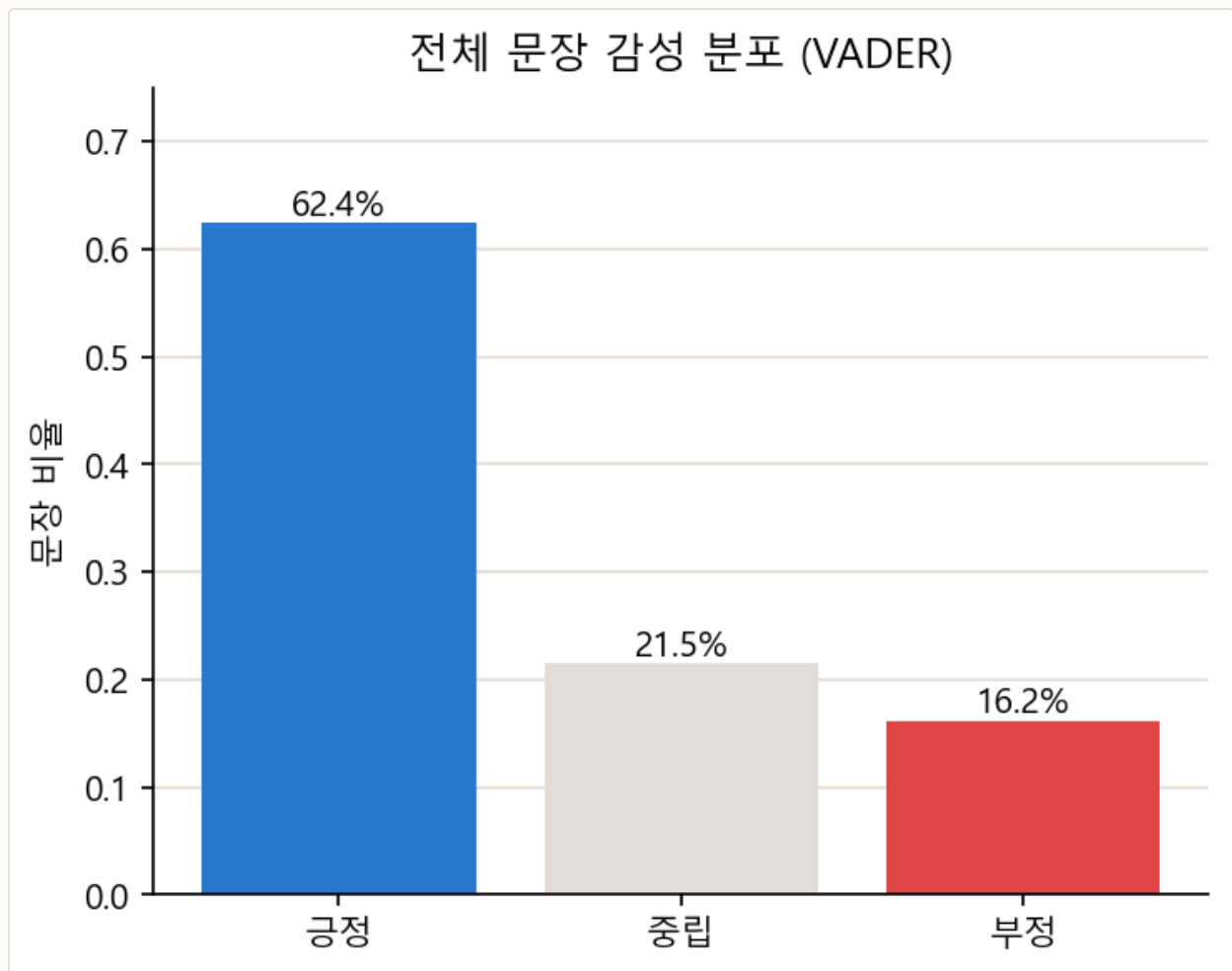


그림 4-3. 전체 문장(7,086개) 감성 분포 — 긍정 62.4%, 중립 21.5%, 부정 16.2%.

해석. 이 데이터셋은 상품·서비스 리뷰 모음이라 전반적으로 긍정 문장이 많다(62.4%). 다만 **부정 문장도 16.2%(약 1,145개)**로 무시할 수 없는 비중이다. 실제 서비스 운영 관점에서는 각 군집(상품군)별로 부정 비율이 높은 세부 주제(aspect)를 추려내면, "어떤 측면에서 불만이 집중되는지"까지 파악할 수 있다 — 이는 군집화와 감성 분석을 결합해야만 나올 수 있는 인사이트로, 감성 점수 하나만 보거나 군집 결과 하나만 보는 것보다 실무적 가치가 크다.

4장 종합

실루엣 점수 기준 최고 조합은 **Count + K-means(0.5489)**지만, 대리 정답 기준 외부 평가와 군집 키워드 확인까지 종합하면 **TF-IDF 기반 파이프라인(특히 SVD n_components=5)**이 **상품군을 더 정확하고 안정적으로 반영**한다. 이는 **내부 지표(실루엣)**와 **외부 지표(ARI/NMI)**가 **항상 같은 결론을 가리키지는 않는다**는 것을 보여주는 사례이며, 비지도 학습에서는 가능한 한 여러 지표를 함께 보는 것이 중요하다는 근거가 된다.

05 Mercari Price Suggestion

중고거래 상품 가격 예측 — 회귀, 대규모 텍스트·범주형 피처

5.1 프로젝트 개요

상품명·설명·브랜드·카테고리·상태·배송 여부가 주어졌을 때 판매 가격(price)을 예측하는 **회귀** 문제다. 목표 변수가 연속적인 실수값이라 분류가 아닌 회귀로 정의된다. 담당 모델은 **LinearRegression**이며, Ridge·Lasso·RandomForest·XGBoost와 비교했다.

1,482,535

학습 데이터 행 수

8 → 44,941

원본 컬럼 수 → 벡터화 후 피처 차원

42.67%

brand_name 결측 비율

0.4818

RMSLE (LinearRegression, 채택)

5.1.1 입력 피처 구성

#	컬럼명	설명	타입
1	name	상품명(텍스트)	Text
2	item_condition_id	상품 상태 — 1(최상)~5(최하)	Ordinal
3	category_name	대/중/소 분류가 "/"로 결합	Text/Cat
4	brand_name	브랜드명 — 결측 다수	Categorical
5	shipping	배송비 부담 주체 — 1:판매자, 0:구매자	Binary
6	item_description	상품 상세 설명(텍스트)	Text
-	price	예측 대상 종속변수(달러)	Target

5.1.2 EDA 요약 통계

`shipping`은 구매자 부담(0) 819,435건, 판매자 부담(1) 663,100건으로 비교적 균형 잡혀 있다.

`item_condition_id`는 1(최상) 640,549건이 가장 많고 5(최하)는 2,384건에 불과해, 대부분 상태가 양호한 중고품이다.

5.1.3 결측치 현황

컬럼	결측치 건수	비율	처리 필요성
category_name	6,327	0.43%	낮음
brand_name	632,682	42.67%	매우 높음
item_description	6	<0.01%	무시 가능

참고: `item_description`에는 실제 결측(NaN)은 6건뿐이지만, "No description yet"이라는 placeholder 문자열이 82,489건 있다 — 형식적으로는 결측이 아니지만 실질적으로는 정보가 없는 값이다.

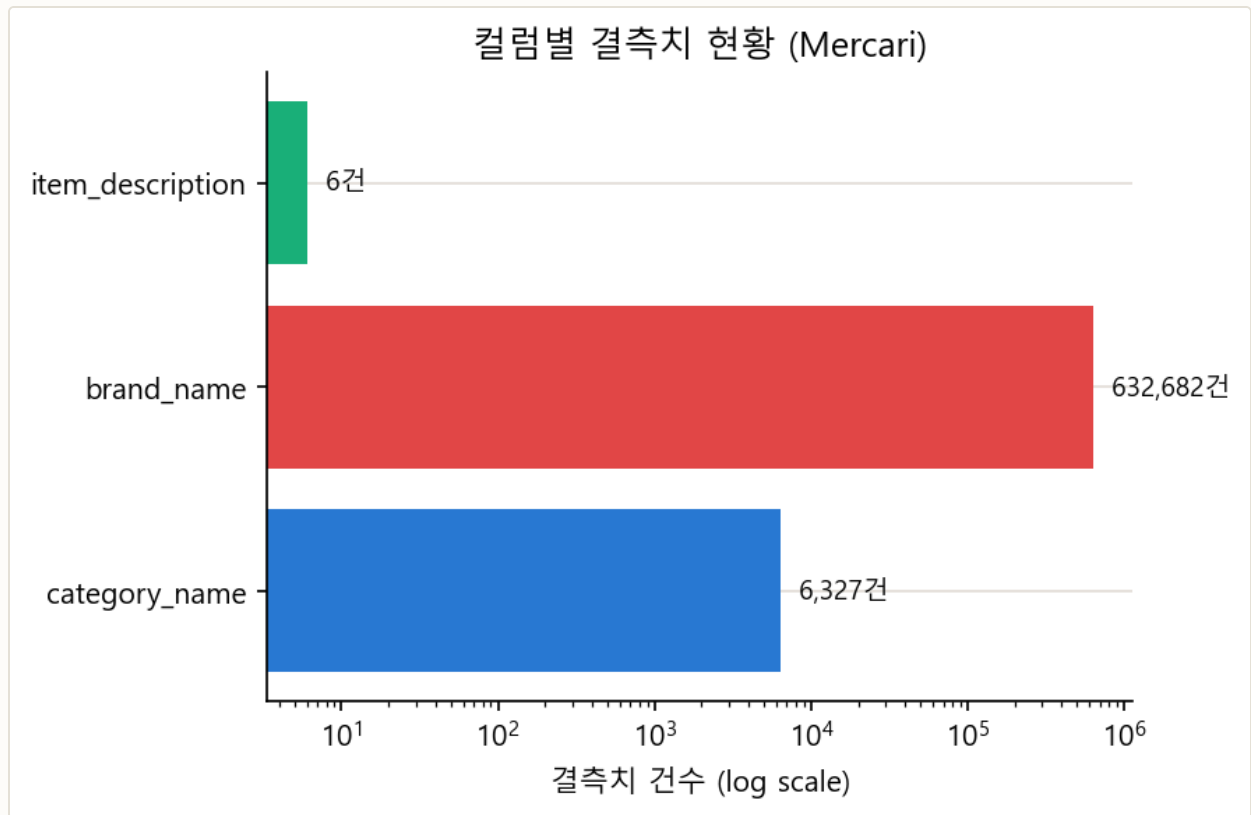


그림 5-1. 컬럼별 결측치 건수(log scale). `brand_name`의 결측(632,682건)은 `category_name`(6,327건)보다 두 자릿수, `item_description`(6건)보다 다섯 자릿수 많다.

로그 스케일로 봐야 하는 이유. 세 컬럼의 결측 건수를 선형 축으로 그리면 `brand_name` 막대만 보이고 나머지 두 컬럼은 눈에 띄지 않을 만큼 차이가 크다(약 100배~10만 배). 로그 스케일은 이 격차를 한 그래프 안에서 동시에 읽을 수 있게 해준다. **`brand_name`만 42.67%가 비어 있다는 것은 "결측치가 있다"는 사실보다 "브랜드 정보 없이도 가격을 맞춰야 하는 상품이 전체의 절반 가까이"라는 의미로 읽어야 한다** — 이 문제를 그대로 두면 브랜드 프리미엄이 실제로 존재하는데도 모델이 이를 절반의 데이터에서만 학습하게 되므로, \$5.3의 브랜드 역추론이 필요한 이유가 된다.

5.1.4 Feature 이름 특징

`category_name` 은 `Women/Tops/T-shirts` 처럼 대/중/소 분류가 "/"로 묶여 있다. 대분류만도 11종 (Women 664,385 / Beauty 207,828 / Kids 171,689 / Electronics 122,690 / Men 93,680 / Home 67,871 / Vintage & Collectibles 46,530 / Other 45,351 / Handmade 30,842 / Sports & Outdoors 25,342 / Other_Null 6,327)이며, 중분류 114종·소분류 871종으로 계층이 깊다.

5.1.5~5.1.6 동일 값 · 중복 Feature

원본 8개 컬럼은 각각 서로 다른 정보를 담고 있어 분산 0이거나 완전 중복인 컬럼은 없다. 다만 `category_name` 과 파생 컬럼 `cat_so` (소분류, 871종)는 `cat_dae` · `cat_jung` 과 계층적으로 겹치는 정보가 많아, 선형 모델의 계수 추정이 불안정해지지 않도록 `cat_so` 는 최종 피처에서 제외했다(\$5.4).

5.1.7 타깃 변수(price) 분포 분석

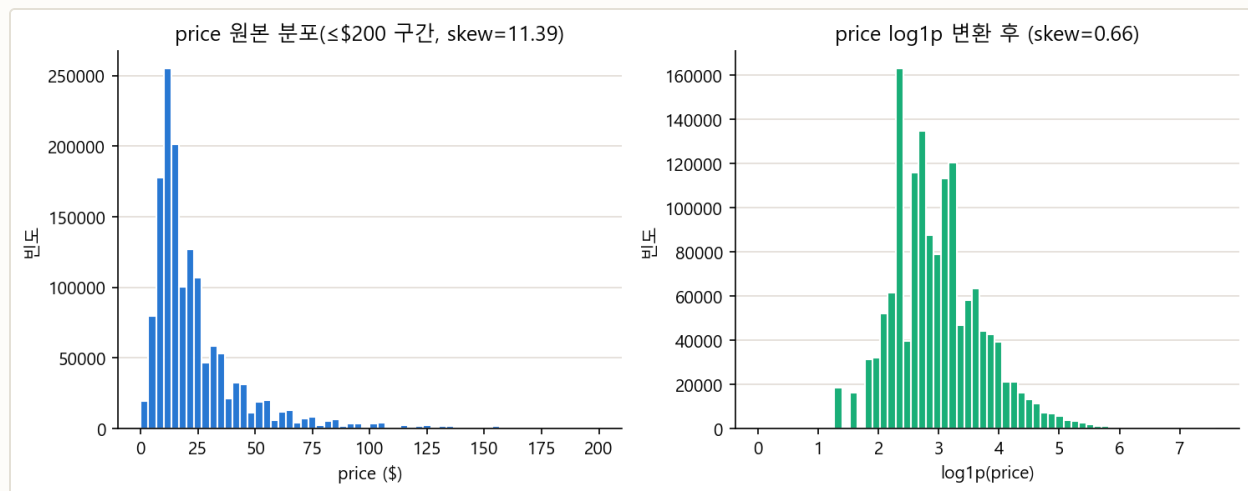


그림 5-2. price 원본 분포($\leq \$200$ 구간, 왜도 11.39) vs $\log_1 p$ 변환 후(왜도 0.66).

해석. 평균과 중앙값의 차이가 크고 고가 상품이 소수 존재해 오른쪽으로 긴 꼬리를 가진 분포다. $\log_1 p$ 변환 이후 왜도가 11.39 $\rightarrow$ 0.66으로 크게 완화되며, 이는 종속변수를 `log1p(price)` 형태로 학습에 사용하는 근거가 된다(\$5.3).

5.2 결과 지표

5.2.1 결과 지표 정의 및 선정 근거 — RMSLE

RMSLE (Root Mean Squared Logarithmic Error)

$$\text{RMSLE} = \sqrt{\text{mean}((\log_1 p(\text{pred}) - \log_1 p(\text{actual}))^2)}$$
 — 예측값과 실제값에 각각 $\log_1 p$ 를 취한 뒤 오차를 제곱평균제곱근한 값이다. 가격이 \$0일 때 $\log(0)$ 이 정의되지 않는 문제를 막기 위해 항상 +1을 더한다.

RMSLE를 선택한 이유는 두 가지다. 첫째, 절대 오차가 아닌 **비율 오차**를 평가한다 — \$10짜리 상품을 \$20으로 예측한 오차(2배)와 \$100짜리 상품을 \$110으로 예측한 오차(1.1배)는 절대 오차가 똑같이 \$10이지만, RMSLE는 전자를 훨씬 크게 벌점을 준다. 저가 상품이 많은 이 데이터셋에서는 이 비율 감각이 훨씬 중요하다. 둘째, **실제보다 낮게 예측했을 때 더 큰 벌점**을 준다 — 예를 들어 실제 \$100인 상품을 \$50으로 예측하면 RMSLE 기여분이 약 0.69이지만, \$150으로 예측하면 약 0.40에 그친다(동일한 \$50 오차인데도).

중고 거래 플랫폼 관점에서의 의미

판매자에게 가격을 **너무 낮게 추천**하면 실제 손해(제값을 못 받음)로 이어지고, **너무 높게 추천**하면 거래가 안 되는 정도의 문제로 그친다. RMSLE는 이 비대칭을 그대로 반영해, 저가 예측에 더 민감한 지표로 작동한다.

5.2.2 결과 지표 해석 계획

학습은 `log1p(price)` 기준으로 진행하고, 평가할 때는 `expm1`로 원래 달러 단위로 복원한 뒤 RMSLE를 계산한다. 이렇게 해야 "학습 목표"와 "최종 평가 지표"가 같은 로그 공간에서 일관되게 유지된다.

5.3 데이터 전처리 & 클리닝

대상	처리
price	<code>log1p(price)</code> 변환 후 학습에 사용
category_name	"/" 기준으로 cat_dae/cat_jung/cat_so 3단 분할
brand_name	결측치 보완(아래) 후 나머지는 "Other_Null"로 채움
item_description	결측 6건을 "Other_Null"로 채움

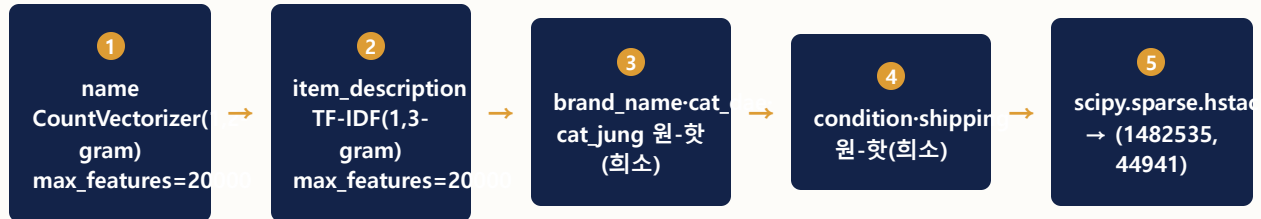
```
known_brands = set(mercari_df.loc[mercari_df['brand_name'].notnull(),
                                'brand_name'].unique())

def find_brand_in_name(name, brand_set):
    words = str(name).split()
    for size in (3, 2, 1):          # 긴 조합부터 우선 매칭 (예: "Kate Spade" > "Kate")
        for i in range(len(words) - size + 1):
            candidate = ' '.join(words[i:i+size])
            if candidate in brand_set:
                return candidate
    return None

# 브랜드 결측(보완 전): 632,682건 → name에서 역추론으로 채운 행: 148,714건
# 역추론 후에도 남은 결측(Other_Null로 채워질 행): 483,968건
```

브랜드 역추론의 효과. brand_name이 결측인 632,682건 중, 상품명(name)에 이미 기존 브랜드명이 등장하는 경우를 찾아 148,714건(23.5%)을 보완했다. 이 방식은 **새로운 브랜드 카테고리를 만들지 않고** 기존 brand_name 값 중에서만 채우므로, 이후 원-핫 인코딩 시 차원 수가 늘어나지 않으면서도 정보 손실을 줄이는 효율적인 방법이다.

5.4 피처 구조화 & 엔지니어링



`name`은 평균 길이가 짧아 단어 등장 빈도로 변별하기 어려우므로 단순 등장 횟수(CountVectorizer)를 사용했다. `item_description`은 문서 전반에 공통적으로 쓰이는 단어가 많아, 핵심 단어에 가중치를 주는 TF-IDF를 사용했다. 범주형 변수(`brand_name`, `item_condition_id`, `shipping`, `cat_dae`, `cat_jung`)는 `LabelBinarizer(sparse_output=True)`로 희소 원-핫 인코딩했다 — 브랜드처럼 종류가 많은 변수를 밀집(dense) 배열로 만들면 메모리가 급격히 늘어나기 때문이다.

파라미터 선택 근거

`max_features=20000`은 TruncatedSVD로 후처리 축소하는 대신 벡터화 단계에서 바로 차원을 통제 한 것이다. 값을 높이면 표현력은 늘지만 메모리·연산 부담과 과적합 위험이 커지고, 낮추면 가볍지만 드물게 등장해도 중요한 단어 정보를 잃을 수 있다. `item_description`의 `ngram_range=(1,3)`은 "not good"처럼 단어 조합이 담는 문맥 정보를 살리기 위한 선택이다 — 범위를 (1,1)로 좁히면 이런 문맥 정보를 잃는다.

5.5 모델링 & 하이퍼파라미터 튜닝

기본 비교

모델	조건	RMSLE
LinearRegression (채택)	item_description 포함	0.4818
LinearRegression	item_description 제외	0.5154
Ridge	alpha=5 (5개 값 중 최선)	0.4792
Lasso	alpha=0.01 (5개 값 중 최선)	0.6924
RandomForest	n_estimators=100, max_depth=15	0.7063

모델	조건	RMSLE
XGBoost	n_estimators=200, max_depth=6, 40만 행 서브샘플	0.5251

item_description을 제외하면 RMSLE가 0.4818→0.5154로 악화된다. 텍스트 설명에 담긴 정보(재질, 상태, 브랜드 언급 등)가 실제로 가격 예측에 기여한다는 뜻이다. **Lasso(alpha=0.01)의 RMSLE 0.6924는 LinearRegression보다 오히려 나쁘다** — L1 정규화가 계수를 0으로 밀어내는 특성상, alpha가 0.1 이상이면 거의 모든 계수가 0이 되어 절편만 남는 상태로 수렴했다(실행 기록상 alpha≥0.1에서 RMSLE가 0.7511로 동일하게 수렴). 44,941차원처럼 극도로 넓고 희소한 피처 공간에서는 Lasso의 강한 피처 선택이 오히려 유용한 신호까지 함께 지워버린 것으로 판단한다.

Optuna 하이퍼파라미터 탐색

Ridge·Lasso·RandomForest·XGBoost에 대해 20만 행 샘플로 Optuna 탐색을 수행한 뒤, 찾은 최적 하이퍼파라미터로 전체 데이터를 재학습해 최종 RMSLE를 산출했다(모델 간 공정한 비교를 위해 최종 비교는 항상 전체 데이터 기준).

모델	탐색 파라미터	전체 재학습 RMSLE
Ridge	alpha≈5.59	0.4792
Lasso	alpha≈0.0001	0.5439
RandomForest	n_estimators=124, max_depth=10, min_samples_leaf=1	0.7202
XGBoost	탐색된 파라미터, 40만 행 서브샘플 재학습	0.5251

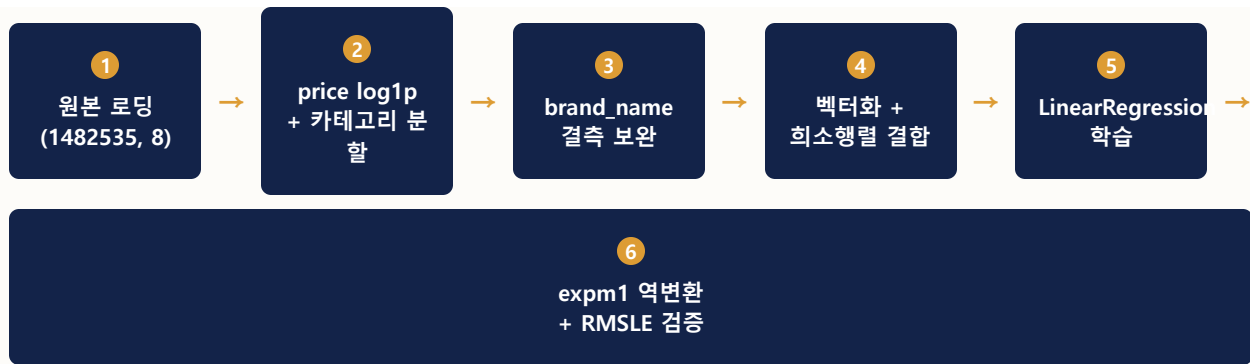
서브샘플링이 필요했던 이유

XGBoost의 tree_method='hist'는 내부적으로 (행×열) 크기의 밀집 bin 버퍼를 만드는데, 44,941열 × 148만 행 전체를 그대로 넣으면 약 53GB 메모리가 필요해 bad_malloc 오류가 발생한다. 다른 모델처럼 전체 행을 쓰는 대신 40만 행으로 서브샘플링해 메모리를 안전한 범위로 제한했다 — 이는 **정확도보다 실행 가능성을 우선한 현실적 제약**이며, XGBoost의 RMSLE(0.5251)가 Ridge(0.4792)보다 나쁜 데는 이 서브샘플링으로 학습 데이터가 148만 건에서 40만 건으로 줄어든 영향도 있을 수 있다.

RandomForest는 Optuna 튜닝 후에도(0.7202) 기본 설정(0.7063)보다 오히려 나빠졌다. 탐색 공간을 n_estimators 50~150, max_depth 6~14로 계산 비용 때문에 좁게 제한한 탓에, 이 고차원 희소 데이터에 필요한 트리 깊이·개수를 충분히 탐색하지 못한 것으로 판단한다. 즉 **탐색 범위 자체가 병목**이 될 수 있다는 사례다.

5.6 성능 검증 결과

5.6.1 파이프라인 요약 다이어그램



5.6.2 지표 결과

순위	모델	조건	RMSLE
1	Ridge	alpha=5.59(Optuna)	0.4792
2	LinearRegression (채택)	튜닝 없음(정규화 파라미터 없음)	0.4818
3	XGBoost	Optuna, 40만 행 서브샘플	0.5251
4	Lasso	alpha=0.0001(Optuna)	0.5439
5	RandomForest	Optuna	0.7202

5.6.3 지표 해석

RMSLE 0.4818이 실무적으로 의미하는 것. RMSLE를 "전형적인 상대 오차 폭"으로 근사 환산하면 $e^{0.4818} \approx 1.62$ 다. 즉 모델이 어떤 상품 가격을 **\$20으로 예측했다면, 실제 가격은 대략 \$12.4(=20÷1.62) ~ \$32.4(=20×1.62) 범위**에 있을 가능성이 높다는 뜻이다(전체 오차 분포를 요약한 근사치이며, 개별 예측의 확정 구간은 아니다). 이는 정가가 명확한 신제품 판매에는 부족한 정밀도지만, **애프터마켓에 정가 자체가 불명확한 중고 개인 거래 플랫폼**에서는 "적정 가격대"를 제시하는 참고 지표로 실용적인 수준이다.

§5.5의 item_description 유무 비교(0.4818 vs 0.5154)에서 보듯, **텍스트 정보가 RMSLE를 약 0.033 개선한다** — 위와 같은 근사로 환산하면 예측 오차 범위가 약 3.4%p 정도 좁아지는 효과다. 텍스트가 담긴 상품 설명이 정량적으로도 가격 예측에 기여한다는 근거다.

회귀계수 해석 — 무엇이 가격을 올리고 내리는가

RMSLE는 "오차가 얼마나 큰가"만 말해준다. **어떤 단어·카테고리가 실제로 가격을 끌어올리는지**를 보려면 회귀계수를 직접 확인해야 한다. 다만 LinearRegression은 44,941차원 희소 피쳐 공간에서 정규화가 없어 개별 계수가 불안정하게 흔들릴 수 있으므로(§5.6.4에서 Ridge가 근소하게 더 우수한 이유와 같은 맥락), §5.6.2와 동일한 피쳐 조합·분할로 재학습한 `Ridge(alpha=5)` 계수를 안정적인 대리 지표로 사용했다. 두 모델의 RMSLE 차이가 0.0026에 불과해(0.4818 vs 0.4792), Ridge 계수가 가리키는 방향은 LinearRegression이 학습한 관계와 사실상 같다고 볼 수 있다.

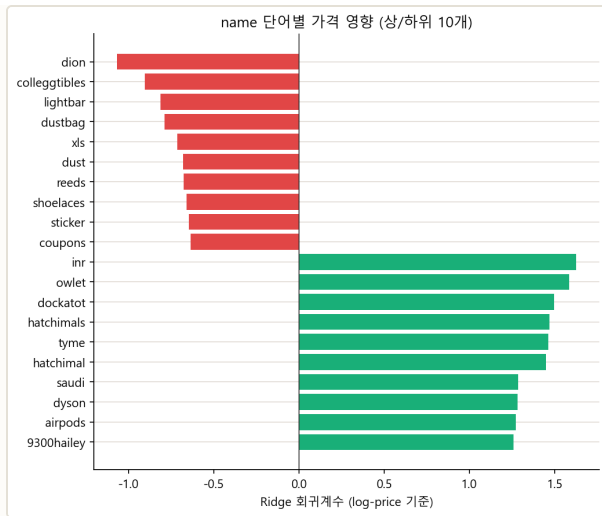


그림 5-3. name에 등장한 단어별 회귀계수(log-price 기준) 상위·하위 10개.

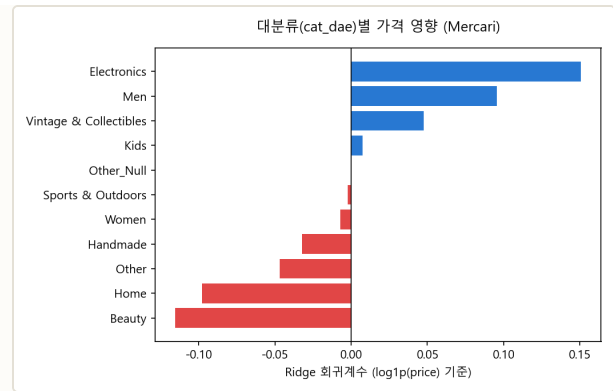


그림 5-4. 카테고리 대분류(cat_dae)별 회귀계수(log-price 기준).

계수를 실제 가격 배율로 환산하면. 종속변수가 `log1p(price)` 이므로 계수는 곱셈(배율)으로 해석해야 한다 — 계수 c 는 다른 조건이 같을 때 가격이 대략 e^c 배가 됨을 뜻한다. 그림 5-3의 상위 10개 중 `owlet` (1.585), `dockatot` (1.495), `hatchimals/hatchimal` (1.468/1.447), `dyson` (1.282), `airpods` (1.273)는 모두 실제로 **고가 프리미엄 브랜드·상품명**이며, 계수를 배율로 바꾸면 대략 **3.6~4.9배** 높은 가격과 연결된다 — 상품명에 이런 단어가 포함되면 모델이 곧바로 "비싼 상품군"으로 인식한다는 뜻이다. 반대로 하위 10개 중 `dustbag` (-0.788), `sticker` (-0.647), `coupons` (-0.636), `shoelaces` (-0.660)는 본품이 아니라 **본품에 딸린 부속품이나 날개로 파는 저가 소모품**을 가리키는 단어들이며, 배율로는 대략 **0.46~0.53배**(원가의 절반 안팎) 수준으로 낮아진다. 같은 그림의 `inr` · `saudi` · `9300hailey` 처럼 단어 자체만으로는 의미를 특정하기 어려운 토큰도 섞여 있는데, 이는 CountVectorizer가 상품명에 붙은 모델 번호·판매자 계정명까지 그대로 하나의 단어로 취급했기 때문으로 보이며, 이런 토큰의 개별 의미를 단정하지는 않는다.

카테고리 대분류(cat_dae) 계수의 의미. `Electronics` (0.151, 약 +16%)와 `Men` (0.096, 약 +10%)은 기준 대비 가격을 끌어올리고, `Beauty` (-0.116, 약 -11%)와 `Home` (-0.098, 약 -9%)은 가격을 끌어내린다. 이는 상식과도 일치한다 — 전자기기는 중고 상태여도 단가 자체가 높은 품목이 많고, 뷰티 제품은 소용량·소모품 위주라 단가가 낮게 형성된다. `Women` (전체의 44.8%를 차지하는 최대 카테고리)은 계수가 거의 0에 가까운데, 데이터에서 가장 큰 비중을 차지하는 카테고리라 모델의 "기준점"에 가장 가깝게 수렴한 것으로 해석한다. **즉 이 회귀계수는 RMSLE 수치 하나로 보이지 않는, "카테고리 가격 프리미엄"이라는 실무적으로 바로 쓸 수 있는 정보를 추가로 제공한다.**

5.6.4 모델별 결과 비교 및 추론

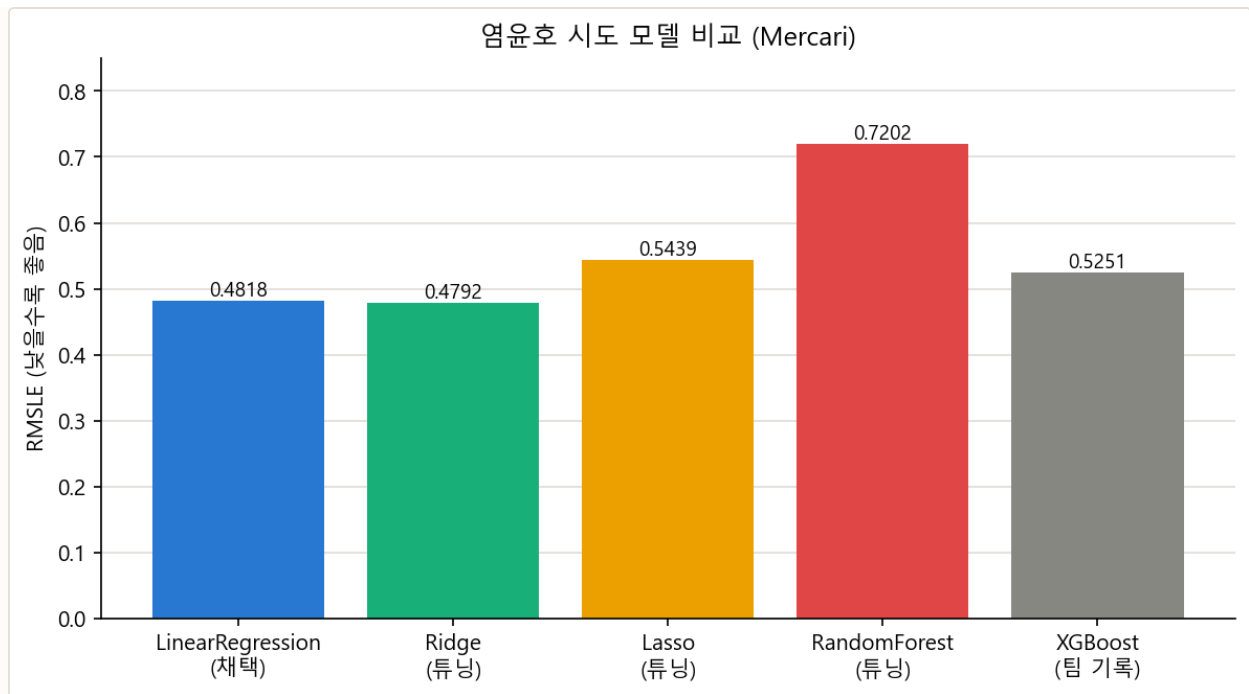


그림 5-5. 담당(염윤호) 시도 모델 비교. XGBoost는 팀 공유 기록(40만 행 서브샘플) 기준.

Ridge(0.4792)가 LinearRegression(0.4818)보다 근소하게 우수한 이유. 44,941차원 중 상당수가 희소(sparse)하고 표본이 적은 단어·브랜드 피쳐다. 정규화가 없는 LinearRegression은 이런 고차원 희소 데이터에서 계수가 불안정해지기 쉬운데, Ridge는 L2 정규화로 계수를 완만하게 수축시켜 이 불안정성을 줄인다. 다만 차이가 0.0026으로 작다는 점은, **이 피쳐 조합에서는 LinearRegression도 이미 충분히 안정적으로 수렴한다**는 뜻이기도 하다.

Lasso·RandomForest·XGBoost가 모두 LinearRegression/Ridge보다 나쁜 이유. Lasso는 강한 피쳐 선택으로 유용한 희소 신호까지 제거했고(\$5.5), RandomForest·XGBoost 같은 트리 기반 모델은 원-핫 인코딩된 초고차원 희소 행렬에서 **분기(split) 후보가 지나치게 많아 최적 분기를 찾기 어렵고**, 트리 개수·깊이를 계산 자원 제약 때문에 충분히 키우지 못했다. 반면 선형 모델은 모든 피쳐에 동시에 가중치를 부여하는 방식이라, 수만 개의 희소한 단어·브랜드 신호를 개별적으로 조금씩 반영하는 이 문제 구조에 구조적으로 더 잘 맞는다.

팀원 비교 — total_result.xlsx 기준, 다른 담당자가 같은 데이터셋에서 얻은 결과다.

담당자	모델	RMSLE	비고
안성준	LightGBM	0.4562	하이퍼옵트 적용
안성준	Ridge	0.4576	-
안성준	XGBoost	0.4822	하이퍼옵트 적용
염윤호(본인)	LinearRegression(채택)	0.4818	-
염윤호(본인)	Ridge	0.4791	Optuna

가장 낮은 RMSLE(안성준, LightGBM 0.4562)에 대한 추론

LightGBM은 트리 기반이지만 **리프 단위(leaf-wise) 성장 방식**과 자체적인 범주형·희소 피처 처리 최적화를 갖추고 있어, 본 파이프라인의 RandomForest·XGBoost보다 원-핫 인코딩된 초고차원 희소 데이터에서 상대적으로 더 효율적으로 분기 후보를 탐색한다. 또한 하이퍼옵트로 체계적인 탐색을 거쳤다는 점도 §5.5에서 지적한 "탐색 범위 부족" 문제를 상대적으로 덜 겪었을 가능성을 시사한다. 다만 본 담당자의 Ridge(0.4791)도 안성준님의 Ridge(0.4576)와 방향성은 비슷하면서 값에 차이가 있어, 전처리 파이프라인의 세부 차이(피처 조합, alpha 탐색 범위 등)가 남아 있음을 보여준다 — 팀 표준 전처리 함수 정립이 필요한 지점이다(§2.3의 팀원별 전처리 차이 문제와 동일한 맥락).

06 회고

공백 — 추후 추가 예정

작성 예정

이 장은 프로젝트 종료 이후 팀 회고를 반영해 추후 작성한다.

07 참고문헌

공백 — 추후 추가 예정

작성 예정

이 장은 인용·참고한 자료 목록을 추후 정리해 작성한다.

